Journal of Statistical Software

MMMMMM YYYY, Volume VV, Issue II. *doi: 10.18637/jss.v000.i00*

rvinecopulib: A Fast and Comprehensive Implementation of Vine Copula Models in C++ and R

Thomas Nagler
Ludwig-Maximilians-Universität München

Thibault Vatter
HES-SO Geneva

Abstract

Vine copulas decompose a multivariate distribution into a cascade of bivariate building blocks and have become a standard tool for flexible dependence modeling in moderate to high dimensions. The R software that made them accessible has not kept pace with the methodology: the package documented in this journal a decade ago has been archived, its successor is in maintenance only, and both confine vine models to continuous data, user-supplied pseudo-observations, a fixed catalogue of parametric pair copulas, and a single structure-selection heuristic. We present **rvinecopulib**, an R package built on **vinecopulib**, a header-only C++17 library. A vine structure is analysed once and the resulting index tells every subsequent algorithm — fitting, density evaluation, simulation, the Rosenblatt transform, and the analytic derivative cascade — exactly which conditional distribution functions it must compute and which it may skip. The package is faster than existing implementations — by about a factor of two on a single core, by four to twelve once threads are used, and by up to three orders of magnitude for nonparametric evaluation — and considerably broader: continuous, discrete, mixed and zero-inflated variables are declared once and handled consistently by every operation; nonparametric pair copulas can be placed at any edge of a vine and evaluated by the same code as any other family; complete multivariate distributions may be built from nonparametric, parametric or user-defined margins supplied through two small S3 protocols; and conditional simulation, conditioning-aware structure selection, automatic truncation, user-supplied tree-selection criteria, analytic scores and Hessians, and observation-specific parameters are all available from the same object model. We describe the design of both the R interface and the C++ engine, validate the implementation against existing software, and report runtime benchmarks.

Keywords: vine copula, pair-copula construction, dependence modeling, nonparametric estimation, discrete data, C++, R, **rvinecopulib**, **vinecopulib**.

1. Introduction

A copula separates a multivariate distribution into its marginal behaviour and its dependence structure (Sklar 1959). In dimensions beyond two or three, however, the catalogue of tractable parametric copulas is thin, and the families that scale — the Gaussian and the Student t — impose strong and often implausible symmetry on the joint tails. Vine copulas resolve this by building a d-dimensional copula out of $d(d-1)/2$ bivariate ones, arranged in a nested sequence of trees (Joe 1996; Bedford and Cooke 2001, 2002). Each bivariate block may be chosen from a different family, so tail asymmetry, tail independence and tail dependence can coexist in the same model. Since the inference algorithms of Aas, Czado, Frigessi, and Bakken (2009) the construction has become a standard tool in finance, insurance, hydrology and the environmental sciences; see Czado (2019) and Czado and Nagler (2022) for book-length and review treatments.

The R software has not kept pace. The only vine copula package ever documented in this journal, **CDVine** (Brechmann and Schepsmeier 2013), was archived from the Comprehensive R Archive Network (CRAN) in 2020. Its successor **VineCopula** (Nagler, Schepsmeier, Stöber, Brechmann, Gräler, and Erhardt 2025) remains widely used but is maintained for compatibility rather than developed, and has never been described in a peer-reviewed software article. Both packages address the same restricted problem: a vine copula fitted to continuous pseudo-observations that the user has computed beforehand, using a fixed catalogue of parametric pair copulas and the greedy tree-by-tree selection of Dißmann, Brechmann, Czado, and Kurowicka (2013). Much of what has been developed for vines since — discrete and mixed margins (Panagiotelis, Czado, and Joe 2012), sparsity-inducing selection criteria (Nagler, Bumann, and Czado 2019), conditional simulation (Cooke, Kurowicka, and Wilson 2015) — has no implementation an ordinary R user can reach, and nonparametric pair copulas (Nagler and Czado 2016) are available only in a separate package (Nagler 2025a) that shares neither the interface nor the engine.

This paper describes **rvinecopulib**, version 1.0.0.1.0, available from CRAN. It is the R interface to **vinecopulib** version 1.0.0, a header-only C++17 library that the package ships in its entirety. The division of labour is worth stating plainly at the outset, because it determines what this paper is about:

> Every algorithm for copulas and vines discussed below is implemented in **vinecopulib**. **rvinecopulib** contributes the R object model, the marginal-modeling layer, and the validation of both. Exactly one estimation algorithm lives in R and not in C++: univariate marginal distribution estimation.

We make two claims for the result.

The first is speed. A vine structure imposes a rigid pattern of reuse: the arguments of a pair copula in tree t are conditional distribution functions produced in tree $t-1$, and depending on the structure a substantial fraction of those are never consumed by any later tree. **vinecopulib** computes this pattern once, when the structure is created, and every subsequent algorithm reads it. Together with a nonparametric estimator whose evaluation cost is independent of the sample size, this is where the package's speed comes from. Section 7 reports the measurements, and the gain depends on what one is doing. Where both implementations spend their time in compiled code running the same optimizer — maximum-likelihood fitting on a single

core — the advantage is about 1.6×. For the operations built on the h-function cascade it is 1.7× to 2.4× when the traversal runs forwards — evaluating a density, a log-likelihood, a Rosenblatt transform — and 1.3× to 1.4× when it runs backwards, as in simulation. What widens the gap is threading, because the engine parallelizes with threads inside one process rather than distributing to worker processes: at $d = 50$ it is 6.3× faster on eight cores than on one, against 2.5×, so the advantage between the two grows from 1.7× to 4.2×. For evaluating a fitted model, where no sequential dependency between trees has to be respected, it reaches 12×. And for nonparametric pair copulas the factors reach three orders of magnitude, which is what turns a nonparametric vine from a demonstration into something one can simulate from.

The second is scope, and it is the claim we expect readers to care about more. **rvinecopulib** handles continuous, integer-valued discrete, and zero-inflated variables in the same model, declared once through a single `var_types` argument and respected by every operation from family selection through conditional simulation. It fits nonparametric pair copulas that behave like any other family: they can be selected, placed at any edge of a vine, and evaluated by the same code. It builds complete multivariate distributions on the original data scale, with margins that may be nonparametric, selected from a parametric catalogue, or implemented by the user against a documented protocol. It performs conditional simulation from an arbitrary conditioning set, selects structures adapted to that conditioning set, and computes analytic scores and Hessians of the likelihood. Table 6 lays out the comparison. No other R implementation offers this combination; outside R the only one that does is **pyvinecopulib** (Vatter and Nagler 2026b), which binds the same engine and is therefore not an independent alternative so much as the same software seen from Python.

The paper is organized as follows. Section 2 fixes notation and states the models. Section 3 describes the C++ engine, concentrating on the structure representation that makes the rest fast. Section 4 is the functional core: the three modeling interfaces, their controls, and the treatment of variable types. Section 5 describes the two protocols through which users supply their own marginal models and tree-selection criteria. Section 6 works through two applications, one continuous and high-dimensional, one mixed and moderate-dimensional. Section 7 validates the implementation against existing software and reports runtimes. Section 8 summarizes and states the limitations.

Throughout, `R>` denotes the R prompt. The replication material consists of three scripts: `code.R` reproduces every reported number and every code example, `parity.R` the comparison against **VineCopula** in Section 7.2, and `make_figures.R` the three data figures. Two things are not script output and should not be looked for there: Figures 1 and 2 are diagrams, and Table 6 is compiled from the documentation and source of the packages it compares.

2. Vine copula models

This section fixes notation and states the three models the package implements. It is deliberately brief; readers wanting a full development should consult Czado (2019) or Joe (2014).

2.1. Copulas and vines

A copula is a multivariate distribution with standard uniform margins. By Sklar (1959), a random vector $\boldsymbol{X} \in \mathbb{R}^d$ has joint distribution F with univariate margins $F_1, \ldots, F_d$ if and

only if there is a copula C with

$$F(x_1, \dots, x_d) = C\{F_1(x_1), \dots, F_d(x_d)\}, \qquad x \in \mathbb{R}^d, \tag{1}$$

and C is unique when the margins are continuous. Differentiating both sides, the joint density is the product of the marginal densities $f_j = \partial F_j / \partial x_j$ and the copula density $c = \partial^d C / \partial u_1 \cdots \partial u_d$,

$$f(x_1, \dots, x_d) = c\{F_1(x_1), \dots, F_d(x_d)\} \times \textstyle\prod_{j=1}^d f_j(x_j). \tag{2}$$

Taking logarithms, the joint log-likelihood separates into a marginal and a copula part. This is what licenses the two-step estimator (Genest, Ghoudi, and Rivest 1995; Joe and Xu 1996; Joe 2005): estimate the margins, then estimate the copula from the pseudo-observations $\widehat{U}_{ij} = \widehat{F}_j(X_{ij})$. Section 4.5 describes the interface that performs both steps in one call, and Section 4.7 the price the second step pays for the first.

Following Joe (1996), Bedford and Cooke (2001) and Bedford and Cooke (2002), any copula density factorizes into $d(d-1)/2$ bivariate conditional copula densities. The order of conditioning is organized by the following structure.

Definition 1. *A* regular vine *is a sequence of trees (V_t, E_t), $t = 1, \dots, d-1$, with node sets V_t and edge sets E_t, such that (i) $V_1 = \{1, \dots, d\}$, (ii) $V_t = E_{t-1}$ for $t \geq 2$, and (iii) whenever two nodes of (V_{t+1}, E_{t+1}) are joined by an edge, the corresponding edges of (V_t, E_t) share a node.*

Conditions (i) and (ii) alone define a vine; condition (iii), the *proximity condition*, is what makes it regular. All vines in this paper are regular, and we drop the qualifier from here on. Each edge $e \in E_t$ carries a label $\{j_e, k_e \mid D_e\}$ with a *conditioned set* $\{j_e, k_e\} \subset \{1, \dots, d\}$ of size two and a *conditioning set* $D_e \subset \{1, \dots, d\} \setminus \{j_e, k_e\}$ of size $t-1$; see Czado and Nagler (2022) for the precise construction. Figure 1 shows an example on four variables. A *vine copula* assigns a bivariate copula $C_{j_e,k_e|D_e}$, called a *pair copula*, to each edge, and the copula density then factorizes as

$$c(\boldsymbol{u}) = \prod_{t=1}^{d-1} \prod_{e \in E_t} c_{j_e,k_e|D_e}(u_{j_e|D_e},\, u_{k_e|D_e} \mid \boldsymbol{u}_{D_e}), \tag{3}$$

where $u_{j_e|D_e} = C_{j_e|D_e}(u_{j_e} \mid \boldsymbol{u}_{D_e})$ is the conditional distribution function of U_{j_e} given $\boldsymbol{U}_{D_e}$, and likewise for k_e. Decomposing a Gaussian copula in this way, for instance, yields Gaussian pair copulas whose parameters are the partial correlations $\rho_{j_e,k_e;D_e}$.

Two consequences of the proximity condition organize everything that follows. First, the arguments $u_{j_e|D_e}$ for an edge in tree $t+1$ can be computed recursively from the pair copulas of trees $1, \dots, t$. Writing

$$h_{j|k}(u_j, u_k) = C_{j|k}(u_j \mid u_k) = \frac{\partial}{\partial u_k} C_{j,k}(u_j, u_k) \tag{4}$$

for the *h-function* of a bivariate copula, each argument in tree $t+1$ is an h-function of a pair copula in tree t, evaluated at arguments produced in tree $t-1$. Estimation therefore alternates between fitting the pair copulas of a tree and evaluating their h-functions to obtain the next tree's data (Aas *et al.* 2009). Second, the recursion is rigid: which h-function each

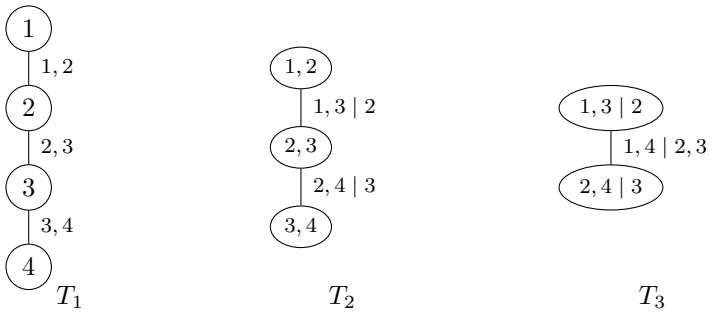

Figure 1: A vine on four variables. Nodes of tree T_{t+1} are edges of T_t, and edges are labelled by their conditioned and conditioning sets. Each edge carries one pair copula, so this vine has $4 \cdot 3/2 = 6$ of them.

edge consumes is fixed by the structure, not by the data. Section 3.1 shows what **vinecopulib** does with that fact.

We follow the common practice of assuming that $c_{j_e,k_e|D_e}(u, v \mid \boldsymbol{u}_{D_e})$ does not depend on $\boldsymbol{u}_{D_e}$, the *simplifying assumption*, which makes (3) a finite-dimensional model. Nagler (2025f) reviews its consequences; **rvinecopulib** does not fit non-simplified vines, and we return to this in Section 8.

2.2. Discrete and mixed variables

When X_j is not continuous, (1) no longer determines C uniquely, and the density in (3) must be replaced by a difference. Write $u_j = F_j(x_j)$ and $u_j^- = F_j(x_j^-)$ for the right and left limits of the marginal distribution function. For a bivariate block in which both variables are discrete, the contribution to the likelihood is the probability that the copula assigns to a rectangle,

$$\frac{C(u_1, u_2) - C(u_1^-, u_2) - C(u_1, u_2^-) + C(u_1^-, u_2^-)}{(u_1 - u_1^-)(u_2 - u_2^-)}, \tag{5}$$

and when only the second is discrete it is a difference of an h-function,

$$\frac{h_{1|2}(u_1, u_2) - h_{1|2}(u_1, u_2^-)}{u_2 - u_2^-}. \tag{6}$$

The h-functions themselves acquire analogous forms when the conditioning variable is discrete. Propagating this through (3) requires carrying, alongside each conditional distribution function $C_{j|D}(u_j \mid \boldsymbol{u}_D)$, its value at the left limit of the *conditioned* variable, $C_{j|D}(u_j^- \mid \boldsymbol{u}_D)$. The likelihood theory is due to Panagiotelis *et al.* (2012) and its use for model selection to Panagiotelis, Czado, Joe, and Stöber (2017); Stöber, Hong, Czado, and Ghosh (2015) treat the mixed discrete–continuous case. Section 4.4 describes how the package asks the user for this information — once — and what it does with it.

We treat a third variable type: a continuous variable with an atom at zero, for which $P(X_j = 0) > 0$ and X_j is continuous on $(0, \infty)$. Such variables are ubiquitous in insurance and hydrology. They are handled by the same left-limit machinery, with $F_j(x_j^-) = F_j(0) - P(X_j = 0)$ at zero and $F_j(x_j^-) = F_j(x_j)$ elsewhere; Funk, Ludwig, Küchenhoff, and Nagler (2026) give a recent application of zero-inflated vines to climate model output.

2.3. Vine distributions

Combining (1) with (3) gives a model for the joint distribution of $\mathbf{X}$ on its original scale: a vine copula for the dependence, and d univariate distributions for the margins. We call this a *vine distribution*. Estimation proceeds in two steps: fit the margins, transform the data to the copula scale by the fitted marginal distribution functions, then fit the vine copula. This is the two-step estimator of Section 2.1: inference for margins (Joe and Xu 1996; Joe 2005) when the margins are parametric, and the semiparametric pseudo-likelihood estimator of Genest *et al.* (1995) when they are estimated nonparametrically, which is the default. Its asymptotic properties for vines are studied by Haff (2013) and, in growing dimension, by Gauss and Nagler (2026).

3. The vinecopulib engine

rvinecopulib ships **vinecopulib** in its entirety as a header-only C++17 library, compiled into the package at installation time. The same headers are used, unmodified, by the Python interface **pyvinecopulib** (Vatter and Nagler 2026b); **rvinecopulib** vendors the upstream headers unmodified, so the two interfaces compute identical numbers whenever they ship the same engine version. External dependencies are obtained through R: linear algebra from **Eigen** (Guennebaud, Jacob *et al.* 2010) via **RcppEigen** (Bates and Eddelbuettel 2013), graph algorithms from **Boost.Graph** (Siek, Lee, and Lumsdaine 2002) and quadrature and special functions from **Boost.Math**, weighted rank correlations from **wdm** (Nagler 2025e), and threading from **RcppThread** (Nagler 2025b). The R–C++ boundary is **Rcpp** (Eddelbuettel and François 2011) and consists of 33 functions, each of which converts an R list to a C++ object, makes one library call, and converts back.

This section describes the parts of the design that a reader needs in order to understand why the workflows of Section 4 are computationally feasible. It is not an API reference; the library documentation at `https://vinecopulib.github.io/vinecopulib/` serves that purpose.

3.1. Structure representation and the computation index

The core data structure is the regular vine. **vinecopulib** stores it in *natural order*: the anti-diagonal of the R-vine matrix is held separately as a permutation `order`, and the remaining entries are relabelled so that the anti-diagonal of the relabelled array reads $1, \dots, d$. The off-diagonal entries live in a triangular array whose row t has $d - t$ entries. Truncation is then a resize of that array: it is $O(1)$ and it frees the memory of the discarded rows, which matters when d is large and the fitted model is sparse.

The reason for this layout is not storage but computation. Consider evaluating (3). Tree t needs, for each of its edges, two conditional distribution functions produced in tree $t-1$. Two facts about that requirement are fixed by the structure alone, independently of the data and of the pair copulas:

1. **Which stored value each edge reads.** For edge e in tree t, one argument is always the h-function computed at position e of the previous tree; the other comes from a position determined by a running column minimum of the structure array. **vinecopulib** precomputes that minimum array once and every algorithm indexes into it directly, replacing a search with a lookup.

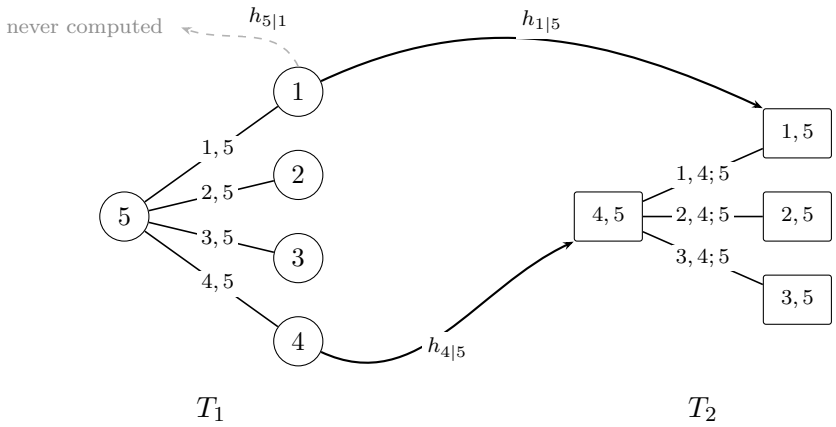

Figure 2: The computation implied by a five-dimensional C-vine, showing its first two trees. Nodes of T_2 are edges of T_1. Each edge of T_1 could supply two conditional distribution functions, but T_2 reads only one of them; the other, drawn dashed, is never evaluated. For a C-vine this holds at every edge of every tree, so exactly half of the h-function evaluations are avoided. Only two of the four transfers are drawn.

2. **Which h-functions are needed at all.** Each edge of tree t can produce two h-functions, (4), but a given one is consumed by a later tree only if the structure says so. **vinecopulib** precomputes two boolean masks, one per h-function, and every algorithm guards its computation on them.

How much this saves depends on the structure, and it is worth being precise because the saving is often assumed to be larger than it is. Writing the fraction of h-functions that some later tree consumes as ϕ, a C-vine has $\phi = 1/2$ exactly in every dimension, so half the work is avoided. For regular vines drawn uniformly at random we measure $\phi \approx 0.58$ at $d = 5$, 0.61 at $d = 10$, 0.60 at $d = 20$ and 0.57 at $d = 50$, a saving of roughly two fifths. A D-vine is the unfavourable case: ϕ rises from 0.67 at $d = 5$ to 0.96 at $d = 50$, because in a path each interior edge feeds two successors. The masks therefore help most for the structures that selection actually produces and least for D-vines in high dimensions, where the benefit of the index is the avoided bookkeeping rather than avoided arithmetic. The replication script reproduces these figures.

The consequence is that a vine structure carries with it a complete index of the computation it implies. *Every* algorithm in the library — the density, model fitting, simulation, the Rosenblatt transform and its inverse, and the analytic derivative cascade of Section 3.5 — traverses the vine with the same two-line dispatch and the same guards. This has two benefits beyond speed. Peak memory is $O(nd)$ rather than $O(nd^2)$, because the working matrices hold one column per edge of the current tree rather than the whole cascade; and adding a new algorithm means writing the per-edge arithmetic, not re-deriving the traversal. Figure 2 illustrates the resulting pattern for $d = 5$.

3.2. Pair copulas

A pair copula is represented by a class `Bicop` that holds a pointer to a family implementation, a rotation, and the types of its two variables. Family implementations expose only stateless

evaluation leaves — density, distribution function, the two h-functions and their inverses — and are written for unrotated, continuous arguments. Three concerns are handled once, above them:

Rotation. Data are rotated before reaching the family and the result is mapped back afterwards, so no family implements rotations. Adding a rotation-aware family costs nothing.

Variable types. The differences (5) and (6) are formed by the layer above the family, from calls to the same unrotated leaves. No family implements discreteness.

Parameter broadcasting. Every leaf receives its parameters as an $m \times p$ matrix with $m \in \{1, n\}$: either one parameter vector broadcast to all observations, or one row per observation. A model whose parameters vary with covariates or with time (Patton 2006; Almeida and Czado 2012; Vatter and Nagler 2018) is therefore evaluated by exactly the same code as a fixed one, at no cost to the fixed case.

Thirteen families are implemented; Table 2 lists them. All are standard (Joe 2014) except the extreme-value family of Tawn (1988). Adding a parametric family requires supplying the family's own mathematics and one line in a factory. For the two largest sub-hierarchies the mathematics needed is small: an Archimedean family is determined by its generator, its inverse and its derivative, and an extreme-value family by its Pickands dependence function and two derivatives; distribution function, density, h-functions and their inverses are inherited. If the family does not supply analytic derivatives, central finite differences are used automatically, so a new family works with the machinery of Section 3.5 immediately.

One numerical detail is worth recording because it accounts for a measurable part of the timings in Section 7. Elliptical pair copulas evaluate the standard normal quantile function once per observation per edge, and it dominates their cost. **Eigen** ships a vectorized implementation of that function but does not advertise it as available for double precision, so the natural array expression silently falls back to a scalar evaluation per element. **vinecopulib** drives the vectorized kernel directly, which agrees with the scalar version to about 2×10^{-15} and roughly halves the cost of a Gaussian pair-copula density.

3.3. Nonparametric pair copulas

The nonparametric family `"tll"` is a transformation local-likelihood estimator. Data are mapped to the standard normal scale by the marginal quantile function, a local polynomial of degree zero, one or two is fitted to the log-density there by weighted maximum likelihood, and the result is mapped back. The transformation idea is due to Geenens (2014); the local-likelihood estimator with degrees one and two is that of Geenens, Charpentier, and Paindaveine (2017), and Nagler, Schellhase, and Czado (2017) add the local-constant variant and study all three in the vine context. The bandwidth is a plug-in rule based on the sample covariance on the normal scale, scaled by a factor depending on the polynomial degree and adjusted for the strength of dependence.

Two implementation choices matter for what follows.

First, the fitted density is stored on a 30×30 grid, equally spaced on the normal scale, and evaluated by interpolation. Evaluation cost is therefore independent of the sample size —

a fitted nonparametric pair copula costs the same to evaluate as a parametric one, which is what makes nonparametric vines tractable in the first place. The grid supports not only interpolation but also the conditional distribution function and its inverse, obtained from cumulative row integrals and from a closed-form root of the resulting piecewise quadratic. These replace, respectively, a repeated numerical integration and a bisection, and are the source of the largest single speed difference reported in Section 7.

Second, the estimator reports an *effective* number of parameters: the sum over the observations of the influence of each point on its own fit (Loader 1999, §5.3.2). This puts the estimator on the same numerical scale as a parametric family's parameter count, so that the two can be compared under one information criterion (Nagler *et al.* 2017), and is what allows `"tll"` to appear in the default candidate set rather than being selected by a separate rule. It is a practical device rather than a theorem: the criteria retain no exact justification once a nonparametric candidate enters, and Section 6.1 gives a case where the nonparametric family is never selected at all.

3.4. Structure selection

Structure selection follows Dißmann *et al.* (2013). Trees are grown one at a time; within a tree, all edges permitted by the proximity condition are enumerated, each is weighted by a measure of dependence between its pseudo-observations, and a maximum spanning tree is extracted. The trees are **Boost.Graph** adjacency lists (Siek *et al.* 2002) and the spanning tree is computed by Prim's or Kruskal's algorithm, or — for the randomized selection of Vatter and Nagler (2025) — by Wilson's loop-erased random walk (Wilson 1996).

Two aspects are worth noting. Edge weights are computed in parallel across candidate edges and pair copulas are fitted in parallel across the edges of a tree, with each fit's own thread budget divided down so that the two levels do not oversubscribe. And pseudo-observations are freed as soon as the tree that produced them has been consumed, which is what keeps peak memory at $O(nd)$.

When the truncation level or the dependence threshold is to be selected rather than fixed, the search is organized as an outer loop over decreasing thresholds with the modified BIC of Nagler *et al.* (2019) as the criterion. Pair copulas whose data have not changed between iterations are not refitted.

3.5. Derivatives

Scores and Hessians of the vine log-likelihood are computed by the cascade of Stöber and Schepsmeier (2013), using the bivariate derivative formulas of Schepsmeier and Stöber (2014). Because a pair copula in tree t takes as arguments h-functions that themselves depend on the parameters of trees $1, \dots, t-1$, differentiating the joint likelihood requires propagating derivatives forward through the same cascade that produces the density.

vinecopulib builds a derivative cache in a single forward pass over the vine, guarded by the same masks as Section 3.1, and the gradient, the joint Hessian and the stepwise Hessian are then read from it rather than recomputed. Closed-form derivatives are implemented for every parametric family, including the two-parameter BB families and the three-parameter Tawn family, for which they were obtained by symbolic differentiation of the generator and Pickands calculus and code-generated. Families without closed forms fall back to central finite

Level	Data scale	Construct	Fit
Bivariate copula	$[0,1]^2$	`bicop_dist()`	`bicop()`
Vine copula	$[0,1]^d$	`vinecop_dist()`	`vinecop()`
Vine distribution	$\mathbb{R}^d$	`vine_dist()`	`vine()`

Table 1: The three modeling interfaces. Objects returned by a fitting function inherit from the corresponding constructed class, so evaluation, simulation and plotting behave identically whether a model was specified or estimated.

differences. For the nonparametric family no derivatives are implemented at all: parameter derivatives are not defined, and rather than silently differentiating an interpolant the library refuses the request.

3.6. Correctness

Numerical claims in a library of this size need evidence, and **vinecopulib** maintains three kinds. Golden-value tests pin quantities that have closed forms — Kendall's τ of each family, quasi-random number streams, bivariate normal and t distribution functions — against references computed to fifty digits. Cross-library parity tests call **VineCopula** from the C++ test suite as a numerical oracle for bivariate densities, distribution functions, h-functions, inverse h-functions, Kendall's τ, Blomqvist's β and tail dependence coefficients, and for vine gradients and Hessians. And a parity harness compares the complete numerical output of two builds of the library against a tolerance schedule that records, for each quantity, the tolerance applied and the reason it was relaxed. Section 7.2 reports what these tests show at the R level.

4. Modeling with rvinecopulib

The package exposes three modeling interfaces, one per level of the construction of Section 2. Each comes as a pair: a constructor that builds a model from known components, and a fitting function that selects one from data. Table 1 summarizes them.

The inheritance in the caption is a deliberate simplification of the user's mental model: `bicop()` returns an object of class `c("bicop", "bicop_dist")`, so every method written for a specified model also applies to a fitted one. Fitted objects carry the additional information a fit produces — the log-likelihood, the number of observations, the controls used — and support the standard generics `print()`, `summary()`, `predict()`, `fitted()` and `logLik()`, and the two copula levels additionally have `plot()` and `contour()` methods. Because `logLik()` returns a proper `"logLik"` object with a degrees-of-freedom attribute, `AIC()` and `BIC()` work without further code.

4.1. Bivariate copulas

A fixed pair copula is built from a family name, a rotation and a parameter vector:

```
R> library("rvinecopulib")
R> cop <- bicop_dist("bb1", rotation = 90, parameters = c(3, 2))
R> cop
```

```
Bivariate copula ('bicop_dist'): family = bb1, rotation = 90,
parameters = 3, 2, var_types = c,c
```

The distribution functions follow R naming conventions: `dbicop()`, `pbicop()` and `rbicop()` for the density, distribution function and random generation, and `hbicop()` for the h-functions (4) and their inverses. The dependence summaries are `par_to_ktau()`, `ktau_to_par()`, `blomqvist_beta()` and `tail_dep()`.

Fitting is done by `bicop()`, whose signature is

```
R> args(bicop)

function (data, var_types = c("c", "c"), family_set = "all", par_method = "mle",
    nonpar_method = "quadratic", mult = 1, selcrit = "aic", weights = numeric(),
    psi0 = 0.9, presel = TRUE, allow_rotations = TRUE, keep_data = FALSE,
    cores = 1)
```

The `family_set` argument accepts individual family names or one of several collective names — `"parametric"`, `"nonparametric"`, `"archimedean"`, `"elliptical"`, `"ev"`, `"itau"`, `"all"` — with partial matching. The default `"all"` includes the nonparametric family; as explained in Section 3.3, its effective degrees of freedom make this a coherent default rather than an arbitrary one. Setting `par_method = "itau"` estimates by inversion of Kendall's τ and restricts the candidate set accordingly, with a warning if the user's set contained families for which this is impossible. Setting `presel = TRUE` discards candidates whose symmetry properties are contradicted by the data before any of them is fitted.

Table 2 lists the implemented families.

4.2. Vine structures

A vine structure is built from a list of tree arrays by `rvine_structure()` and from an R-vine matrix by `rvine_matrix()`. The two special cases are built from a variable order by `cvine_structure()` and `dvine_structure()`. The functions `as_rvine_structure()` and `as_rvine_matrix()` convert between the two representations, and `rvine_structure_sim()` draws one at random.

```
R> structure <- dvine_structure(order = c(3, 1, 4, 2))
R> as_rvine_matrix(structure)

4-dimensional R-vine matrix ('rvine_matrix')
1 4 2 2
4 2 4
2 1
3
```

`truncate_model()` sets all pair copulas above a given tree to independence. It has methods for structures, matrices, vine copulas and vine distributions, and for fitted models it adjusts the recorded number of parameters and log-likelihood accordingly.

Family	`family`	Par.	Rot.	Tail dep.	Deriv.
Independence	`"indep"`	0	—	none	yes
Gaussian	`"gaussian"`	1	—	none	yes
Student t	`"t"`	2	—	both	yes
Clayton	`"clayton"`	1	all	lower	yes
Gumbel	`"gumbel"`	1	all	upper	yes
Frank	`"frank"`	1	—	none	yes
Joe	`"joe"`	1	all	upper	yes
BB1	`"bb1"`	2	all	both	yes
BB6	`"bb6"`	2	all	upper	yes
BB7	`"bb7"`	2	all	both	yes
BB8	`"bb8"`	2	all	upper†	yes
Tawn	`"tawn"`	3	all	upper, asym.	yes
Transf. local likelihood	`"tll"`	eff.	—	data-driven	no

Table 2: Implemented pair-copula families. Rotations are by 90, 180 and 270 degrees; families marked — are invariant under them. The nonparametric family reports an effective number of parameters (Loader 1999, §5.3.2). Analytic derivatives are used by `scores()` and `hessian()`; where they are unavailable the package falls back to finite differences, except for the nonparametric family, where derivatives are refused. †BB8 is tail independent over the interior of its parameter space; upper tail dependence arises only in the boundary case $\delta = 1$.

4.3. Vine copulas

`vinecop()` selects and fits a vine copula. Its arguments fall into four groups, and it is more useful to describe the groups than to list them.

What the pair copulas may be: `family_set`, `par_method`, `nonpar_method`, `mult`, `presel` and `allow_rotations`, all with the meaning they have in `bicop()`.

How the structure is chosen: `structure` accepts a fixed structure, a partially specified one — a k-truncated structure fixes the first k trees and leaves the rest to be selected — or `NA` for full selection. `tree_crit` selects the edge weight, and `tree_algorithm` the spanning-tree algorithm, with `"mst_prim"` and `"mst_kruskal"` giving the maximum spanning tree and `"random_weighted"` and `"random_unweighted"` drawing one at random (Vatter and Nagler 2025).

How large the model may be: `trunc_lvl` truncates the vine in the sense of Brechmann, Czado, and Aas (2012) and `threshold` sets pair copulas with weak dependence to independence. Each accepts a fixed value, or `NA` to have it selected by the modified vine Bayesian information criterion (mBICV) of Nagler *et al.* (2019), available as `selcrit = "mbicv"` and as the function `mBICV()`, whose prior probability of non-independence is controlled by `psi0`. `selcrit` chooses the criterion used for families.

Everything else: `weights`, `cores`, `keep_data`, `show_trace`, and `vinecop_object`, which refits the parameters of an existing model while holding its structure and families fixed.

A typical call selects everything:

```
R> u <- pseudo_obs(returns)
R> fit <- vinecop(u, trunc_lvl = NA, selcrit = "mbicv", cores = 4)
R> head(summary(fit), 4)
```

```
 tree edge conditioned conditioning var_types family rotation
    1    1      35, 38                    c,c      t        0
    1    2       19,  4                     c,c      t        0
    1    3      39, 42                    c,c      t        0
    1    4       34,  4                     c,c      t        0
  parameters df  tau loglik
0.34, 11.02  2 0.22     92
 0.69,  4.71  2 0.48    494
 0.71,  5.25  2 0.50    546
 0.74,  3.67  2 0.53    627
```

The `summary()` method returns a data frame with one row per pair copula, so it can be filtered and sorted like any other; printing it also appends a legend mapping variable indices to names. The fitted model is inspected through a family of accessors: `get_structure()`, `get_matrix()`, `get_pair_copula()`, `get_family()`, `get_parameters()` and `get_ktau()` for a single edge, and `get_all_pair_copulas()` and its relatives for all of them at once.

4.4. Variable types

Discrete and mixed data are the capability that most sharply distinguishes **rvinecopulib** from the alternatives, and the design decision that makes them usable is that the user declares the types once.

At the copula level, types are declared through `var_types`, a character vector with one entry per variable, `"c"` for continuous and `"d"` for discrete. Evaluating (5) and (6) requires both $F_j(x_j)$ and $F_j(x_j^-)$, so the data matrix must carry both. Two layouts are accepted. The *expanded* layout has $2d$ columns: the first d hold $F_j(x_j)$ and the second d hold $F_j(x_j^-)$, with the entries for continuous variables ignored. The *compact* layout has $d + k$ columns where k is the number of discrete variables, and appends left limits only for those. The two are interchangeable everywhere — in `bicop()`, `vinecop()`, the Rosenblatt transform and conditional simulation — and the package converts between them internally. Note that the types belong to the model, not to the call: they are given once to `vinecop_dist()` or `vinecop()`, and every later evaluation reads them from the fitted object.

```
R> model <- vinecop_dist(pair_copulas, dvine_structure(1:3),
+    var_types = c("c", "d", "c"))
R> x <- cbind(rnorm(500), rpois(500, 1), rnorm(500))
R> u_expanded <- cbind(
+    pnorm(x[, 1]), ppois(x[, 2], 1), pnorm(x[, 3]),
+    pnorm(x[, 1]), ppois(x[, 2] - 1, 1), pnorm(x[, 3]))
R> u_compact <- u_expanded[, c(1, 2, 3, 5)]
R> all.equal(dvinecop(u_expanded, model), dvinecop(u_compact, model))

[1] TRUE
```

At the vine-distribution level the left limits are computed by the package from the fitted margins, so the user supplies data on its original scale and declares types with a single top-level `var_types` argument to `vine()`. Three types are recognized there: `"c"`, `"d"` and `"zi"`

for a continuous variable with an atom at zero. Types may also be carried by the data itself — an ordered factor is treated as discrete, and a column wrapped in `zero_inflated()` as zero-inflated — in which case an explicit `var_types` that contradicts the data is an error rather than a silent override. Unordered factors are expanded into indicator variables.

4.5. Vine distributions and margins

`vine()` fits a complete multivariate distribution. Marginal and copula behaviour are controlled by two lists:

```
R> fit <- vine(
+    dat,
+    margins_controls = list(family_set = "all", selcrit = "bic"),
+    copula_controls = list(family_set = "parametric", trunc_lvl = NA),
+    var_types = c("c", "d", "zi"),
+    cores = 4)
```

`margins_controls` takes three entries. `family_set` names the marginal candidates: `"kde1d"` for the nonparametric estimator of Nagler and Vatter (2025), any distribution supported by **univariateML** (Moss 2025), the aliases `"nonparametric"`, `"parametric"` and `"all"`, or a user-defined family object as described in Section 5.1. Candidates may be given per variable, as a list indexed positionally or by variable name, and each entry may itself be a vector of candidates. `selcrit` chooses among them by log-likelihood, the Akaike information criterion (AIC) or the Bayesian information criterion (BIC), with candidates whose supported types exclude the variable's declared type discarded before fitting. `cores` controls parallel marginal fitting, which uses forked processes and is therefore serial on Windows.

Marginal distribution estimation is the one algorithm in the package implemented in R rather than C++. Fitted margins are returned in `fit$margins` and can be examined with the generics of Section 5.1. One detail is worth knowing when doing so: an ordered factor is fitted on its integer codes shifted to start at zero, so a factor with levels $1, \ldots, 6$ yields a margin supported on $\{0, \ldots, 5\}$. `margin_info()` reports that support, and `rvine()` maps simulated values back to the original levels automatically; only direct calls to `dmargin()`, `pmargin()` and `qmargin()` see the internal coding.

4.6. Simulation, conditioning and transforms

`rvinecop()` and `rvine()` simulate from a fitted model, on the copula and the data scale respectively. Both accept `qrng = TRUE` for quasi-random rather than pseudo-random draws (Cambou, Hofert, and Lemieux 2017), and both stay synchronized with R's `set.seed()`.

Both also perform *conditional* simulation. The values are supplied through `u_cond` or `x_cond` and the variables being conditioned on through `conditioning_set`, which accepts indices or names:

```
R> sim <- rvine(1000, fit, x_cond = c(veh_value = 1.5, agecat = 3),
+    conditioning_set = c("veh_value", "agecat"))
```

A single vector of conditioning values is recycled to all n draws; an n-row matrix gives observation-specific conditioning, which is what a predictive distribution over a set of covariate profiles requires. The model is reoriented transiently for the purpose and is not modified.

Conditional simulation is efficient when the conditioning variables occupy the tail of the vine's sampling order, because they then form a self-contained sub-vine. `vinecop()` therefore accepts a `conditioning_set` argument of its own, and `vine()` accepts one as an entry of `copula_controls`; it biases structure selection so that the resulting model has this property while remaining otherwise optimal in the sense of Dißmann *et al.* (2013). The algorithm is described in the companion paper (Nagler and Vatter 2026); the underlying idea of conditionalizing a regular vine is due to Cooke *et al.* (2015), and a restricted version for C- and D-vines to Bevacqua, Maraun, Haff, Widmann, and Vrac (2017), whose construction is implemented in the Python package **VineCopulas** (Claassen, Koks, de Ruiter, Ward, and Jäger 2024) from which our implementation was adapted.

`rosenblatt()` and `inverse_rosenblatt()` implement the transform of Rosenblatt (1952), which maps a sample from the model to independent uniforms and back, and which is the basis of residual diagnostics for vines. Both accept a `conditioning_set`. For discrete variables the transform is not uniquely defined; by default the package randomizes within the atom (Brockwell 2007; Czado, Gneiting, and Held 2009), which yields exactly uniform residuals under a correctly specified model, and `randomize_discrete = FALSE` returns the upper endpoint instead.

4.7. Likelihood derivatives and uncertainty

`scores()` returns the matrix of observation-wise score contributions and `hessian()` the averaged Hessian, for both bivariate and vine copulas. Columns are ordered by tree, then edge, then parameter within the pair copula.

The `step_wise` argument distinguishes two objectives. With `step_wise = TRUE`, the default, the pseudo-observations entering each pair copula are held fixed, which is what the tree-by-tree sequential estimator of Section 2.1 actually solves; the resulting gradient is close to zero at a fitted model. With `step_wise = FALSE` the derivatives propagate through the h-function cascade, giving the gradient of the joint log-likelihood, which at a sequentially fitted model is generally not zero.

From these, `vcov()` assembles a covariance estimate, and `confint()` turns it into Wald intervals:

```
R> S <- matrix(0.4, 4, 4)
R> diag(S) <- 1
R> u <- pseudo_obs(matrix(rnorm(800 * 4), 800, 4) %*% chol(S))
R> fit <- vinecop(u, family_set = "parametric", keep_data = TRUE)
R> sqrt(diag(vcov(fit)))
R> sqrt(diag(vcov(fit, step_wise = FALSE)))
R> confint(fit, level = 0.95)
```

The estimator solves $\bar{s}(\hat{\theta}) = 0$ with $\bar{s}(\theta) = n^{-1} \sum_i s_i(\theta)$, so it is an M-estimator, and the relevant matrices are

$$A = -\frac{\partial \bar{s}(\theta)}{\partial \theta^\top}, \qquad B = n^{-1} \sum_{i=1}^{n} s_i s_i^\top. \tag{7}$$

`vcov()` returns $A^{-1}BA^{-\top}/n$, the misspecification-robust sandwich of White (1982). Rows and columns are named by tree, conditioned pair and conditioning set, so a coefficient can be located in the model without counting.

The choice of `step_wise` changes what A is, and the distinction matters enough to state. With `step_wise = FALSE`, $\bar{s}$ is the gradient of the joint log-likelihood and A is minus its averaged Hessian: symmetric, and the classical setting in which $A^{-1}BA^{-\top}$ reduces to the familiar $A^{-1}BA^{-1}$. With `step_wise = TRUE` the pseudo-observations entering each pair copula are held fixed. That is the estimating equation sequential estimation actually solves, but it is not the gradient of any objective: A is a Jacobian rather than a Hessian, and it is block triangular rather than symmetric. The sandwich formula still applies — which is why (7) is written with $A^{-\top}$ on the right — but the information equality $A = B$ does not, so the simpler A^{-1}/n would be wrong here. That is why the sandwich is the only form the package offers.

`vcov()` treats the copula data as observed, so it quantifies uncertainty in the pair-copula parameters *given* the margins. This is the usual two-stage approximation (Joe 2005; Ko and Hjort 2019), and its cost is measurable. The replication material reports a small experiment: a three-dimensional Gaussian D-vine with equicorrelation 0.5, $n = 1000$, 1000 replications, so that a single coverage proportion carries a Monte Carlo standard error of 0.007. Nominal 95% intervals attained an average coverage of 0.956 when the true margins were used and 0.929 when they were estimated by ranks; the paired difference is 0.027 with a standard error of 0.004. The individual per-parameter proportions are 0.957, 0.957, 0.954 and 0.940, 0.906, 0.941 respectively, but with three parameters and this much Monte Carlo error we draw no conclusion about which of them suffers most. What the experiment supports is the aggregate statement: ignoring marginal estimation makes the intervals too short, by a small but clearly measurable amount.

That is why `vcov()` and `confint()` have methods for `bicop` and `vinecop` objects but none for `vine` objects. Correcting the two-stage approximation requires the influence function of the marginal estimator, and Section 5.1 makes that estimator a user-supplied function: there is nothing the package can differentiate. A method that silently conditioned on the fitted margins would be anticonservative by the amount just measured, so we provide none, and document the bootstrap instead. The asymptotics of the step-wise estimator in growing dimension are studied by Gauss and Nagler (2026).

Lower-level derivative access is available where it is needed to build something else. `dbicop()` and `hbicop()` take a `deriv` argument naming first- or second-order partial derivatives with respect to the arguments or the parameters, and `dvinecop(keep_all = TRUE)` returns the per-edge densities and h-functions that the density computation produced anyway. Finally, `dbicop()`, `pbicop()`, `hbicop()`, `rbicop()` and `dvinecop()` accept a `parameters` argument holding one parameter row per observation, which is what downstream packages modeling covariate-dependent dependence (Vatter and Chavez-Demoulin 2015; Vatter and Nagler 2018) require from a vine backend.

5. Extending rvinecopulib

Two parts of the modeling pipeline are open to user code: the marginal models and the criterion by which vine trees are selected. Both are specified as protocols — small sets of generic functions that a user implements for their own class — rather than as fixed catalogues

of options. The pattern follows **ROI** (Theußl, Schwendinger, and Hornik 2020) and the **sandwich** decomposition of covariance estimators into `bread()` and `estfun()` (Zeileis 2006).

5.1. The marginal protocols

Marginal models enter a vine distribution in two roles: as a *fitted margin*, which must be evaluated, and as a *family*, which must be fitted to data before it can be evaluated. The two roles have separate protocols.

A fitted margin implements four generics. Three evaluate the distribution — `dmargin()`, `pmargin()` and `qmargin()`, dispatching on the margin rather than on the data — and the fourth, `margin_info()`, reports metadata: the family name, the variable type, the support, the number of parameters, and the log-likelihood. The evaluation methods must be vectorized and must propagate `NA`.

The type reported by `margin_info()` determines how the package computes the left limit that Section 2.2 requires:

$$F_j(x^-) = \begin{cases} F_j(x), & \text{type "c",} \\ F_j(x-1), & \text{type "d",} \\ F_j(0) - P(X_j = 0) \text{ at } x = 0, \quad F_j(x) \text{ otherwise}, & \text{type "zi".} \end{cases} \tag{8}$$

This is the load-bearing part of the design. Because the left limit is derived from the protocol, a zero-inflated margin participates in an ordinary vine with no special case anywhere in the copula layer: the copula code sees two numbers per variable and does not know or care how they were produced.

A margin *family* implements two generics: `fit_margin(family, x, weights, type)`, which returns an object satisfying the fitted-margin protocol, and `margin_info(family)`, which reports the family name and the variable types it supports. A fitter receives weights on every call and must either use them or reject them explicitly; silently ignoring them would corrupt a weighted fit without warning.

Two family adapters ship with the package: `kde1d_family()` for nonparametric margins, with the bandwidth and boundary options of **kde1d** as arguments of the family rather than as entries of a control list, and `univariateML_family()` for the parametric catalogue of **univariateML**. Two further constructors build *fitted* margins directly, for use when a margin is known rather than estimated: `stats_margin()` wraps a **stats** distribution with fixed parameters, and `margin_dist()` builds one from its density, distribution and quantile functions.

Writing a family is correspondingly small. The following implements a zero-inflated log-normal margin — a plausible model for insurance claim amounts, and one no catalogue provides:

```
R> fit_zilnorm <- function(x, weights = numeric(), type = "zi") {
+    if (length(weights) == 0) weights <- rep(1, length(x))
+    pos <- x > 0
+    p0 <- sum(weights[!pos]) / sum(weights)
+    ml <- sum(weights[pos] * log(x[pos])) / sum(weights[pos])
+    sl <- sqrt(sum(weights[pos] * (log(x[pos]) - ml)^2) / sum(weights[pos]))
+    margin_dist(
```

```
+      d = function(q) {
+        ifelse(q > 0, (1 - p0) * dlnorm(q, ml, sl), ifelse(q == 0, p0, 0))
+      },
+      p = function(q) {
+        ifelse(q > 0, p0 + (1 - p0) * plnorm(q, ml, sl), ifelse(q < 0, 0, p0))
+      },
+      q = function(p) {
+        ifelse(p <= p0, 0, qlnorm(pmax((p - p0) / (1 - p0), 0), ml, sl))
+      },
+      family = "zilnorm",
+      type = "zi",
+      support = c(0, Inf),
+      npars = 3,
+      loglik = sum(weights[!pos]) * log(p0) +
+        sum(weights[pos] * (log(1 - p0) + dlnorm(x[pos], ml, sl, log = TRUE)))
+    )
+  }
R> zilnorm <- margin_family(fit_zilnorm, family_name = "zilnorm", types = "zi")
```

The result is a first-class candidate. It can be used on its own, or placed in a candidate set where it competes against the built-in families under the selection criterion:

```
R> fit <- vine(dat, margins_controls = list(
+    family_set = list(claimcst0 = list(zilnorm, "kde1d"), veh_value = "all"),
+    selcrit = "bic"))
```

5.2. Custom tree-selection criteria

The edge weight used in structure selection is set by `tree_crit`. The built-in choices are Kendall's τ, Spearman's ρ, Hoeffding's D (Hoeffding 1948), the mutual information of a Gaussian copula, and the maximum correlation of Rényi (1959) estimated by the alternating conditional expectations algorithm of Breiman and Friedman (1985). Czado, Jeske, and Hofmann (2013) compare several of these; the differences are generally small, which is a reason to make the choice open rather than to enlarge the catalogue.

A function may also be supplied as `tree_crit`. It is called with a two-column matrix of pseudo-observations and a vector of weights, and must return one finite number, whose absolute value is used as the edge strength. Rows with missing values or zero weight are removed before the call. The example below selects trees by the coefficient of Chatterjee (2021), which detects functional relationships that rank correlations miss. Two details of the contract are worth noticing in it. The coefficient is deliberately asymmetric — it measures how far Y is a function of X — whereas a tree edge is undirected, so the callback symmetrizes by taking the larger of the two directions. And because no weighted version is implemented here, the callback rejects weights rather than silently ignoring them, exactly as a margin fitter must:

```
R> xi_one <- function(x, y) {
+    n <- length(x)
```

```
+    r <- rank(y[order(x)], ties.method = "max")
+    1 - 3 * sum(abs(diff(r))) / (n^2 - 1)
+  }
R> xi <- function(data, weights) {
+    if (length(weights) > 0) stop("xi does not support weights")
+    max(xi_one(data[, 1], data[, 2]), xi_one(data[, 2], data[, 1]))
+  }
R> fit_xi <- vinecop(u, tree_crit = xi, family_set = "parametric")
```

The callback is evaluated on the thread that started the fit, so it need not be thread-safe; pair-copula fitting within a tree remains parallel.

5.3. What is not extensible

Pair-copula families are a closed set. There is no R-level hook by which a user can add one, and we do not consider adding one advisable: a family must supply not only a density but a distribution function, two h-functions, two inverse h-functions, parameter bounds and a Kendall's τ map, all of them called in the innermost loop of every algorithm in the package, and an R callback in that position would dominate the runtime. A new family is a contribution to **vinecopulib**; as Section 3.2 notes, for the Archimedean and extreme-value hierarchies this amounts to a generator or a Pickands function and its derivatives. Users needing data-driven flexibility rather than a specific parametric form are better served by the nonparametric family, and users needing parameters that vary across observations by the `parameters` argument of Section 4.7.

6. Illustrations

We work through two applications. The first is continuous and high-dimensional, and exercises structure selection, sparsity control and the nonparametric families; it is also the data on which Section 7 reports timings, so that those are measured on a real problem rather than a synthetic best case. The second is moderate-dimensional and mixed, and exercises the marginal protocols, variable types, conditional simulation and the uncertainty tooling.

6.1. Financial returns

We begin with the setting vine copulas were developed for. The `SP500_const` data of **qrmdata** record daily closing prices of the constituents of the S&P 500 index. We take the window 2010–2015, keep the 473 series with no missing values, draw a random sample of $d = 50$ of them, and work with the $n = 1509$ daily log-returns.

```
R> data("SP500_const", package = "qrmdata")
R> prices <- SP500_const["2010-01-01/2015-12-31"]
R> prices <- prices[, colSums(is.na(prices)) == 0]
R> tickers <- sort(sample(colnames(prices), 50))
R> returns <- diff(log(prices[, tickers]))[-1, ]
R> u <- pseudo_obs(as.matrix(returns))
```

One caveat before any model. Daily returns are close to uncorrelated but far from independent: they exhibit volatility clustering, and the standard practice is to filter each series with a univariate time-series model and fit the vine to the standardized residuals (Dißmann *et al.* 2013). We work with the raw returns because the object of this section is the copula machinery rather than the marginal dynamics, and nothing below depends on the distinction; a reader building a risk model should filter first.

Fitting all $50 \cdot 49/2 = 1225$ pair copulas without restriction takes about ninety seconds on four cores and yields a model with 452 parameters, of which 361 pair copulas are not the independence copula. The interesting question is whether so large a model is warranted, and the package answers it by selecting the truncation level under the modified BIC:

```
R> fit <- vinecop(u, family_set = "parametric", trunc_lvl = NA,
+    selcrit = "mbicv", cores = 4)
```

Table 3 compares four fits. Truncation removes 53 of the 452 parameters and 39 trees at a cost of 504 in log-likelihood. Figure 3 shows the whole path, and it needs a word of explanation, because reading Table 3 left to right invites a wrong conclusion. The untruncated model has the better modified BIC (-52247 against -51810), yet `trunc_lvl = NA` returned ten trees. This is not a failure of the search: the selection is *sequential*, not a minimization over truncation levels. It grows trees one at a time and stops the first time adding one fails to improve the criterion, exactly as the structure itself is grown, and it never revisits that decision. On these data the per-tree gains are not monotone, so an early flat stretch halts a search that would have improved again later.

Whether that is the behaviour one wants is a modeling question rather than a software one. A sequential rule is cheap and gives a sparse model; a reader who wants the criterion's minimum over the whole path can compute it directly, as Figure 3 does, and pass the result to `trunc_lvl`. What matters for reading the table is that the two rows answer different questions.

The third row of Table 3 adds the nonparametric family to the candidate set and returns exactly the parametric fit: at this dimension and sample size the modified BIC never prefers a nonparametric pair copula at any edge. That is a fact about the software's behaviour, not a verdict on nonparametric estimation, and we return below to the comparison that would support such a verdict. The tree criterion matters less than either: switching Kendall's τ for the maximum correlation moves the log-likelihood by 2 in 27,685.

The selected model is dominated by the Frank copula (162 of the 312 fitted pair copulas), followed by the Student t (66) and Gumbel (32). That the Student t appears mostly in the first tree and the Frank copula mostly later is the familiar pattern: tail dependence is a feature of raw returns, and it is largely accounted for by the time the higher trees are reached.

A word on what these criteria are for, because it is easy to ask more of them than they can deliver. AIC, BIC and the modified BIC compare *parametric* candidates for a given pair copula, using parameter counts that mean the same thing across candidates. A nonparametric estimator reports effective degrees of freedom, which are on a comparable numerical scale but are not parameter counts in the same sense, and the criteria carry no justification for choosing between the two on that basis. The comparison worth making between a parametric and a nonparametric approach is therefore between whole models, and out of sample.

Setting	Trees	Non-indep.	Par.	Log-lik.	BIC	mBICV
Par., untruncated	49	361	452.0	28189	−53069	−52247
Par., mBICV truncation	10	312	399.0	27685	−52449	−51810
All families, mBICV	10	312	399.0	27685	−52449	−51810
Par., mBICV, `mcor`	10	315	401.0	27687	−52438	−51794

Table 3: Fitted vine copulas for 50 S&P 500 constituents, $n = 1509$. "Non-indep." counts pair copulas that are not the independence copula, out of 1225 for the untruncated model and 445 for the truncated ones. The third row adds the nonparametric family to the candidate set and recovers the second row exactly; see the text, and Table 4 for the comparison that should be made instead.

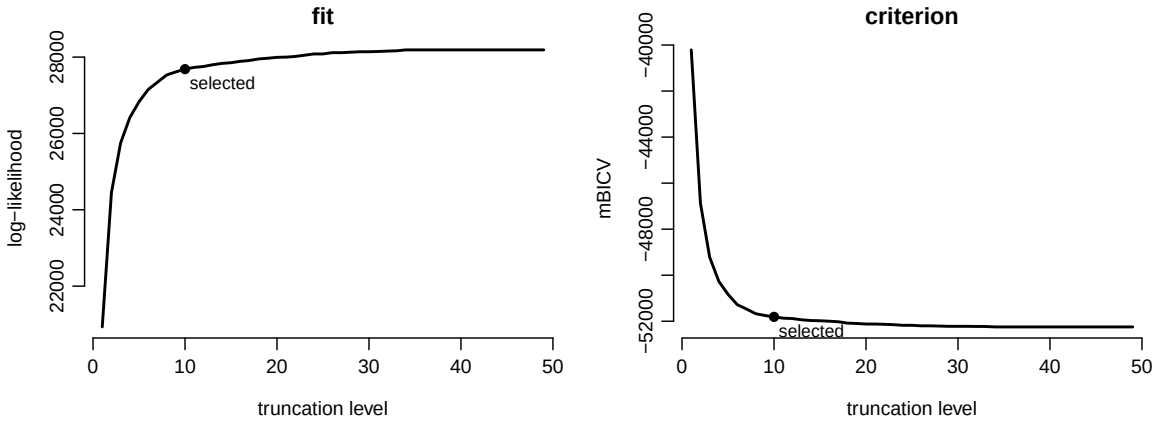

Figure 3: Log-likelihood and modified BIC against truncation level for the returns data, obtained by truncating the untruncated fit at each level. The filled point marks the level chosen by `trunc_lvl = NA`. The criterion continues to improve beyond it, for the reason given in the text.

We split the sample, fit on the first 1000 observations, and evaluate the mean log-density on the remaining 509:

```
R> train <- u[1:1000, ]; test <- u[1001:1509, ]
R> fits <- list(
+    parametric_itau = vinecop(train, family_set = "itau", par_method = "itau",
+      selcrit = "mbicv", trunc_lvl = NA, cores = 8),
+    parametric_mle  = vinecop(train, family_set = "parametric",
+      selcrit = "mbicv", trunc_lvl = NA, cores = 8),
+    nonparametric   = vinecop(train, family_set = "tll", trunc_lvl = 9, cores = 8))
R> sapply(fits, function(m) mean(log(dvinecop(test, m))))
```

Table 4 collects the result, with the independence copula as a baseline. Two things follow.

The first concerns estimation method rather than family set. Fitting by inversion of Kendall's τ instead of maximum likelihood is more than twenty times faster — 1.6 seconds against 37 — and on these data it also predicts *better* out of sample, 16.12 against 15.85. The two-parameter families that maximum likelihood can reach and τ-inversion cannot buy in-sample

Model	Trees	Parameters	Fit (s)	Held-out log-density
Independence	0	0.0	—	0.00
Parametric, `itau`, mBICV	9	337.0	1.6	**16.12**
Parametric, `mle`, mBICV	9	352.0	37.0	15.85
Nonparametric, 9 trees	9	16,818.4	1.8	8.29
Nonparametric, 3 trees	3	7,039.7	0.5	13.33
Nonparametric, 1 tree	1	2,681.9	0.2	11.44

Table 4: Whole-model comparison on the returns data, $d = 50$: fitted on 1000 observations, evaluated by mean log-density on a held-out 509. Parameter counts for the nonparametric models are effective degrees of freedom. Fitting times are on eight cores.

fit that does not survive the split. For exploratory work at this dimension, `par_method = "itau"` is the better default, and it is the setting we use for the timings in Section 7.3.

The second concerns the nonparametric family, and it is a caution. A nonparametric vine truncated at the same level as the parametric one has 16,818 effective degrees of freedom against 1000 training observations, and it predicts far worse: 8.29 against 16.12. Truncating it harder helps — three trees give 13.33, one tree 11.44 — but nothing in this family of models approaches the parametric fit. That is the expected outcome and not a defect: kernel estimators need sample sizes that grow with the number of pair copulas, and $d = 50$ with $n = 1000$ is not that regime. Nonparametric pair copulas earn their place in low dimensions, where a misspecified parametric family costs more than the variance of a kernel estimate; Section 7.3 shows what they cost to evaluate once fitted.

6.2. Insurance claims

We turn to data of a kind that no other vine implementation in R can accommodate. The `dataCar` data of Wolny-Dominiak and Trzesiok (2014) record 67,856 one-year motor insurance policies: the claim amount incurred, the number of claims, the vehicle's value, the fraction of the year the policy was exposed, and categorical descriptions of the vehicle and the driver. We take a random subsample of 10,000 policies.

Four of these variables are not continuous, and they are not discrete in the same way. Claim cost is continuous on $(0, \infty)$ but has an atom at zero of mass 0.929: most policies make no claim. Vehicle age and driver age category are ordered factors. Exposure and vehicle value are continuous. We exclude the claim count, because it is positive exactly when the claim cost is positive; a vine containing both would model a deterministic relationship as a copula, which no family can represent well. Deciding which of two structurally linked variables to keep is an ordinary modeling judgment, and the package will not make it for the user.

Variable types need not be declared separately when the data carry them. An ordered factor is recognized as discrete, and `zero_inflated()` marks a column as continuous-with-an-atom:

```
R> data("dataCar", package = "insuranceData")
R> idx <- sample(nrow(dataCar), 10000)
R> dat <- data.frame(
+    cost      = zero_inflated(dataCar$claimcst0[idx]),
+    veh_value = dataCar$veh_value[idx],
```

```
+    exposure  = dataCar$exposure[idx],
+    veh_age   = ordered(dataCar$veh_age[idx]),
+    agecat    = ordered(dataCar$agecat[idx]))
R> predictors <- c("veh_value", "exposure", "veh_age", "agecat")
```

The claim cost needs a marginal model that no catalogue supplies. We use the zero-inflated log-normal family built in Section 5.1 and let the remaining margins be selected from explicit candidate lists: the two continuous variables from a handful of parametric families and the nonparametric estimator, the two ordered factors from the nonparametric estimator alone. Naming the candidates rather than passing `"all"` is deliberate here. Vehicle value is recorded to the nearest hundred dollars and one policy in the subsample carries a recorded value of exactly zero, which the strictly positive families in the **univariateML** catalogue — the log-normal, gamma, and Weibull among them — cannot represent; naming the candidates keeps the selection over families that can.

```
R> fit <- vine(
+    dat,
+    margins_controls = list(
+      family_set = list(
+        cost = zilnorm,
+        veh_value = c("kde1d", "norm", "sstd", "std", "logis", "cauchy"),
+        exposure = c("kde1d", "beta", "norm", "unif"),
+        veh_age = "kde1d", agecat = "kde1d"),
+      selcrit = "bic"),
+    copula_controls = list(family_set = "parametric", selcrit = "bic",
+                           conditioning_set = predictors, keep_data = TRUE),
+    keep_data = TRUE, cores = 4)
```

The `conditioning_set` argument asks for a structure in which the four policy characteristics occupy the tail of the sampling order. This costs nothing in fit quality — the selected pair copulas are the same — but it is what makes the conditional simulation below reliable rather than a matter of luck. Without it, whether a given conditioning set can be conditioned on depends on the structure the selection happens to return, and a model fitted on a different subsample may refuse.

Table 5 reports what was selected. The user-defined family wins for the claim cost, as it must, being the only candidate that can represent an atom; among the automatically selected margins a skewed Student t is chosen for vehicle value and a beta distribution for exposure, which is supported on $(0, 1)$. The two ordered factors are fitted nonparametrically and report effective parameter counts of 8.28 and 7.56. Figure 4 shows three of the fits.

The selected vine copula is sparse: three of its ten pair copulas are the independence copula, and of the remaining seven only two carry appreciable dependence. The strongest is vehicle value against vehicle age, $\hat{\tau} = -0.517$, fitted by a rotated BB7 copula — older vehicles are worth less, which is not news. The substantively interesting one is exposure against claim cost, $\hat{\tau} = 0.356$, fitted by a Joe copula rotated by 180 degrees. That family is asymmetric with lower tail dependence — here 0.59 — and the asymmetry is the point: the association between exposure and cost is much stronger in the lower tail than the upper. Policies exposed

Variable	Type	Selected family	Parameters	Log-likelihood
`cost`	`"zi"`	`zilnorm`	3.00	−8539.2
`veh_value`	`"c"`	`sstd`	4.00	−13600.9
`exposure`	`"c"`	`beta`	2.00	44.6
`veh_age`	`"d"`	`kde1d`	8.28	−13688.5
`agecat`	`"d"`	`kde1d`	7.56	−17199.9

Table 5: Selected marginal models for the insurance data. Types were inferred from the column classes. Parameter counts for the nonparametric margins are effective degrees of freedom. Exposure has a positive log-likelihood because its density exceeds one on part of the unit interval.

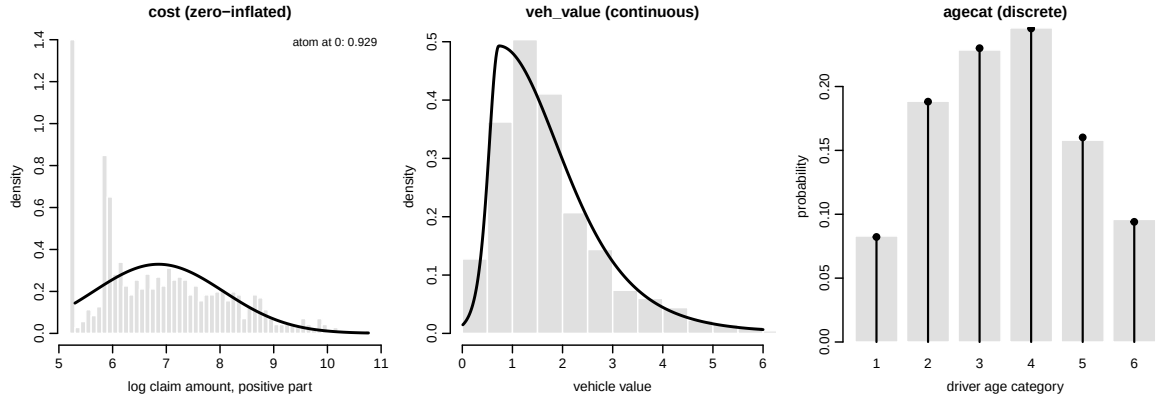

Figure 4: Three of the five fitted margins. Left: the positive part of the claim cost on the log scale, with the fitted zero-inflated log-normal density rescaled to the positive part; the atom at zero carries mass 0.929. Centre: vehicle value with the selected skewed Student t density. Right: the driver age category, with observed relative frequencies as bars and fitted probabilities as points.

for a small fraction of the year almost never claim, far more reliably than heavily exposed policies always do. A symmetric family would have averaged the two and described neither. Everything from tree two onwards is independent or negligible. It is worth noting that the modified BIC would *not* truncate this model — evaluated along the nested path it prefers keeping trees two and three by about 35 units — and that in any case the conditioning-aware selection used here requires an untruncated vine, so the two features cannot presently be combined. Table 6 lists them separately for that reason.

```
R> summary(fit$copula)
```

```
 tree edge conditioned conditioning var_types  family rotation
    1    1        1, 3                   d,c     joe      180
    1    2        5, 2                   d,c clayton       90
    1    3        2, 4                   c,d     bb7      270
    1    4        4, 3                   d,c clayton        0
    2    1        1, 4              3    d,d   indep        0
```

```
   2    2        5, 4            2       d,d clayton       90
   2    3        2, 3            4       c,c     joe      180
   3    1        1, 2         4, 3       d,c   indep        0
   3    2        5, 3         4, 2       d,c   frank        0
   4    1        1, 5      2, 4, 3       d,d   indep        0
 parameters df    tau loglik
          2  1  0.356  136.0
      0.092  1 -0.044   30.0
  1.2, 2.0  2 -0.517 3005.3
      0.057  1  0.028   10.3
             0  0.000    0.0
      0.075  1 -0.036   14.2
          1  1  0.019    4.9
             0  0.000    0.0
       0.25  1  0.028    8.1
             0  0.000    0.0

---
1 <-> cost,   2 <-> veh_value,   3 <-> exposure,   4 <-> veh_age,
5 <-> agecat
```

The `var_types` column shows `"d"` where the data declared `"zi"`. That is the type system working as designed: the third type exists at the vine-distribution level, where it selects the left limit in (8), and the copula layer needs only to know whether a variable has atoms. `vine()` therefore collapses `"zi"` to `"d"` before handing the copula its types, and the zero-inflation is carried entirely by the margin.

Wald intervals for the pair-copula parameters are available even though the model contains discrete variables, where the backend differentiates by finite differences rather than in closed form. They are requested from the copula component rather than from the fitted distribution, which is the interface making explicit that they condition on the margins:

```
R> confint(fit$copula)

             2.5 % 97.5 %
T1.1,3       1.849  2.161
T1.5,2       0.068  0.117
T1.2,4.par1  1.125  1.216
T1.2,4.par2  1.948  2.098
T1.4,3       0.032  0.083
T2.5,4;2     0.046  0.103
T2.2,3;4     1.012  1.056
T3.5,3;2,4   0.128  0.378
```

These intervals treat the fitted margins as known, so they understate the uncertainty by the amount measured in Section 4.7. For this model the effect is likely larger than in that experiment, because the claim cost's margin is a three-parameter fit rather than a rank

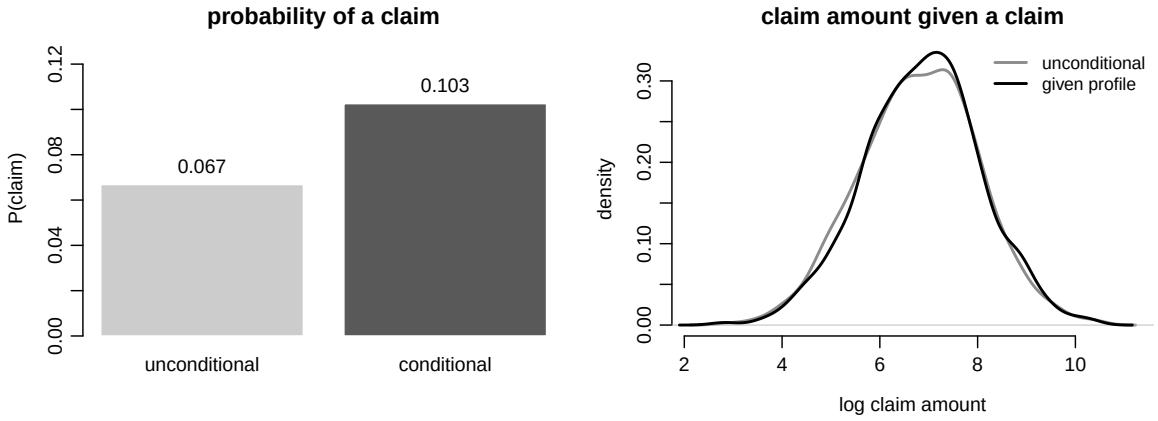

Figure 5: Conditional and unconditional predictions from the fitted vine distribution, from 20,000 draws each. Left: probability that a policy incurs a claim. Right: the distribution of the claim amount given that a claim occurs, on the log scale.

transform. An honest interval for the whole distribution requires a bootstrap over both stages, which the recipe in `?parameter_uncertainty` performs.

The question such a model is fitted to answer is predictive: what claim distribution should we expect from a policy with given characteristics? This is conditional simulation, and it needs no refitting. Conditioning values are supplied on the original data scale, in the units the data came in, and the conditioning variables are named:

```
R> profile <- data.frame(veh_value = 1.5, exposure = 0.75,
+    veh_age = ordered(2, levels = levels(dat$veh_age)),
+    agecat  = ordered(2, levels = levels(dat$agecat)))
R> sim <- rvine(20000, fit, x_cond = profile,
+    conditioning_set = c("veh_value", "exposure", "veh_age", "agecat"))
```

For this profile — a mid-value vehicle, two years old, a young driver, exposed for three quarters of the year — the simulated probability of a claim is 0.103 against an unconditional 0.067, and the expected cost per policy rises from 123 to 196. Figure 5 shows why. The conditioning raises the probability that a claim occurs by half, but the distribution of the amount *given* that one occurs barely moves: its median goes from 934 to 990. Almost all of the change in expected cost is frequency, not severity. That decomposition is what an insurer wants from such a model, and reading it off requires a joint model of a zero-inflated variable and a set of mixed covariates — which is the case this section was chosen to make.

7. Comparison and benchmarks

This section compares **rvinecopulib** with the available alternatives. We report what the package can do that they cannot, then establish that it computes the same numbers where a comparison is possible, and only then report how long it takes. The order is deliberate: a speed claim from an unvalidated implementation is worth nothing.

All measurements were made on an idle AMD Ryzen Threadripper 2990WX with 32 cores and 125 GB of memory, running Ubuntu 22.04 with the reference BLAS, under R 4.6.1 (R Core Team 2026), **rvinecopulib** 1.0.0.1.0, **VineCopula** 2.6.2 (Nagler *et al.* 2025), and **kdecopula** 0.9.3 (Nagler 2018). The **VineCopula** build includes two parallelization fixes we contributed while preparing this comparison, described at the end of Section 7.3; without them its multicore timings measure the defects rather than the algorithm.

Every timing is the fastest of several repetitions rather than the median. Anything else running on the machine can only make a repetition slower, so the minimum is the best available estimate of the uncontended cost, and it also discards the first-call warm-up. This matters more than it may appear: medians of three repetitions moved by up to 37% between two runs of the replication script on the same machine, while minima are stable to a few percent.

7.1. Functionality

Table 6 compares capabilities. We include the two R packages that fit vine copulas, the two that fit nonparametric copulas, the archived predecessor, and the actively maintained implementations in Python and MATLAB.

7.2. Numerical validation

Table 7 compares **rvinecopulib** with **VineCopula** on every quantity the two have in common, for all ten shared parametric families in all admissible rotations — 31 configurations — on a fixed 19×19 grid of the unit square. Densities, distribution functions, h-functions and their inverses, Blomqvist's β and the tail dependence coefficients all agree to between 10^{-16} and 10^{-11}, which is what one expects of two independent implementations of the same closed forms in double precision.

Kendall's τ is the exception, and it is worth being precise about which implementation is responsible. For Frank copulas the two differ by 7.8×10^{-4}, far above rounding. Frank's τ is $1 - 4/\theta + 4D_1(\theta)/\theta$ where D_1 is the first Debye function, which can be evaluated to machine precision by adaptive quadrature. At $\theta = 3$ that reference value is 0.307246959431; **rvinecopulib** returns it to 1.1×10^{-16} and **VineCopula** is off by 7.8×10^{-4}. The remaining discrepancies, of order 10^{-8} for BB6 and BB8, are of the same character and arise from the quadrature used for integrals with an endpoint singularity.

Writing this section also turned up a defect on our side. Joe's Kendall τ,

$$\tau(\theta) = 1 + \frac{2}{2-\theta}\{\psi(2) - \psi(2/\theta + 1)\},$$

has a removable singularity at $\theta = 2$, where numerator and denominator vanish together. The implementation evaluated the quotient directly and returned `NaN` at exactly that point — reachable in practice, since $\theta = 2$ is an ordinary interior point that a user may specify directly or arrive at from a prior fit. The limit is $1 - \psi'(2) = 2 - \pi^2/6$, and the fix is a series expansion in a neighbourhood of the singularity; the entry for Joe in Table 7 is now exact. We mention it because it is the kind of defect that a comparison against an independent implementation finds and a test suite written against one's own implementation does not.

Vine log-likelihoods. For each cell of the runtime grid below we verified that the two packages, given the same data and the same matched settings, select structures with the same log-

	rvinecopulib	VineCopula	kdevine	CDVine	pyvinecopulib	VineCopulas	MATVines
Models							
Regular vines	✓	✓	✓	—	✓	✓	✓
C- and D-vine constructors	✓	✓	✓	✓	✓	✓	✓
Parametric pair copulas	11	12	12	10	11	6	9
Nonparametric pair copulas	✓	—	✓	—	✓	—	—
Margins fitted jointly	✓	—	✓	—	✓	✓	—
User-defined margins	✓	—	—	—	✓	—	—
Data							
Discrete variables	✓	—	—	—	✓	✓	—
Mixed continuous/discrete	✓	—	—	—	✓	✓	—
Zero-inflated variables	✓	—	—	—	✓	—	—
Observation weights	✓	✓	✓	—	✓	—	—
Selection							
Automatic truncation	✓	—	—	—	✓	—	—
Automatic thresholding	✓	—	—	—	✓	—	—
Modified BIC	✓	—	—	—	✓	—	—
User-defined tree criterion	✓	✓	—	—	✓	—	—
Randomized structure search	✓	—	—	—	✓	—	—
Inference and simulation							
Rosenblatt transform	✓	✓	—	✓	✓	✓	✓
Conditional Rosenblatt	✓	—	—	—	✓	—	—
Conditional simulation	✓	—	—	—	✓	✓	—
Conditioning-aware selection	✓	—	—	—	✓	✓	—
Analytic scores and Hessians	✓	✓	—	—	✓	—	—
Standard errors	✓	✓	—	—	✓	—	—
Robust (sandwich) covariance	✓	—	—	—	✓	—	—
Per-observation parameters	✓	—	—	—	✓	—	—
Goodness-of-fit tests	—	✓	—	✓	—	—	✓
Vuong and Clarke tests	—	✓	—	✓	—	—	—
Implementation							
Compiled backend	✓	✓	—	✓	✓	—	—
Multithreading	✓	—	—	—	✓	—	—
On CRAN / PyPI	✓	✓	✓	—	✓	✓	—
Peer-reviewed software reference	—	—	—	✓	—	✓	✓

Table 6: Capability comparison. Counts of parametric pair-copula families exclude the independence copula and count rotations of a family once; they were read off each package's own family enumerator. **VineCopula** counts one more than **rvinecopulib** because it provides two Tawn parameterizations where we provide one. **pyvinecopulib** binds the same engine as **rvinecopulib**, so its column is identical by construction; it is shown so that the Python alternatives can be compared with each other. **CDVine** was archived from CRAN in 2020 and **MATVines** (Coblenz 2021) is distributed outside a package repository. **VineCopula** additionally provides goodness-of-fit, Vuong and Clarke tests and an interactive interface, which **rvinecopulib** does not; see Section 8.

Quantity	Largest absolute deviation
Density	2.0×10^{-14}
Distribution function	5.6×10^{-16}
h-functions	4.1×10^{-15}
Inverse h-functions	3.0×10^{-11}
Blomqvist's β	4.4×10^{-16}
Tail dependence coefficients	0
Kendall's τ, all but Frank	9.1×10^{-8}
Kendall's τ, Frank	7.8×10^{-4}

Table 7: Agreement with **VineCopula** over 31 family-rotation combinations and a 19×19 grid. The Frank entry is discussed in the text.

likelihood. The check is part of the replication script and reports on the very fits that were timed, not on a separate pair fitted for the purpose. Under maximum likelihood the largest relative discrepancy was 1.1×10^{-6}, and the two selected the same number of non-independent pair copulas in every cell — 9 of 10 at $d = 5$, 40 of 45 at $d = 10$, and 154 of 190 at $d = 20$. Under τ-inversion, where near-ties between families are broken differently, the two do not reach the same optimum: the gap is 0.5 at $d = 5$ and grows to 169 at $d = 50$, or 0.05% to 0.6% of the log-likelihood, always in our favour. Section 7.3 returns to what that means for the comparison. One restriction must be stated: the two do not share a convention for the orientation of rotated families, so a vine containing 90- or 270-degree rotations cannot be constructed identically in both. The vine-level comparison is therefore restricted to unrotated and 180-degree families; rotations are covered at the bivariate level, where the correspondence is unambiguous.

Nonparametric estimation. Section 7.3 reports the accuracy comparison against **kdecopula** alongside the timings, because the two cannot be read separately.

Discrete models. No alternative implementation exists, so we validate internally: (5) against direct evaluation of the copula measure on a rectangle, the empirical Kendall's τ of simulated data against the model value, uniformity of the randomized Rosenblatt residuals, and the round trip `inverse_rosenblatt(rosenblatt(u))` to machine precision. The analytic scores of Section 4.7 were checked against central finite differences of the vine log-likelihood, agreeing to a relative 4×10^{-5} on a mixed continuous–discrete model.

7.3. Runtime

How much faster **rvinecopulib** is depends on what is being computed, and the differences are large enough that a single summary figure would be misleading. We separate fitting from evaluation, and within fitting we separate maximum likelihood from τ-inversion.

Matched settings. Against **VineCopula** we match the setting exactly: the candidate set is the intersection of the two catalogues, the selection criterion is AIC, the tree criterion is Kendall's τ, the independence pre-test is disabled, and no truncation is applied. We check in every cell that the two implementations reach a comparable optimum before quoting a ratio. Under maximum likelihood the log-likelihoods agree to a relative 10^{-6} or better. Under τ-

inversion they do not agree exactly, because near-ties between families are broken differently: the relative gap is at most 5.7×10^{-3}, but in absolute terms it reaches 33 at $d = 20$ and 169 at $d = 50$, always in our favour. We report the ratios anyway, because the gap is between 0.05% and 0.6% of the log-likelihood and the two models are of the same size to within a pair copula or two: the counts of non-independent pair copulas agree exactly at $d = 5$ and $d = 10$, and differ by one of 190 at $d = 20$ and by seven of 1225 at $d = 50$. But the direction should be stated: we are fitting a marginally better model in marginally less time, so the τ-inversion rows slightly flatter us, and a reader should treat them as comparing two implementations of the same procedure rather than two computations of the same number.

Fitting. Table 8 reports structure selection and estimation. Two regimes are visible and they say different things.

Under maximum likelihood on a single core the gain is modest and slightly *decreasing* in dimension: 1.66× at $d = 5$, 1.58× at $d = 10$, 1.49× at $d = 20$. This is the honest number for that case and it is not dramatic: both implementations spend most of their time in compiled code running the same optimizer over the same families, and there is little to win.

Under τ-inversion the picture changes, because parameter estimation stops dominating and the cost moves to the parts **vinecopulib** parallelizes — computing edge weights over all admissible candidate edges, and fitting the pair copulas of a tree concurrently. On one core the advantage is remarkably flat at 1.66× to 1.75× across all four dimensions. It is the scaling that differs: on eight cores **rvinecopulib** is 6.3× faster than itself at $d = 50$, **VineCopula** 2.5×, so the advantage between them widens from 1.66× on one core to 4.2× on eight. In absolute terms that is a four-and-a-half-second fit against a nineteen-second one.

The comparison inverts at small dimension, and the table shows it rather than hiding it. At $d = 5$ **VineCopula** is slower on eight cores than on one (1.04 against 0.41 seconds), because a tree of five variables does not contain enough work to repay starting a worker pool and shipping data to it. **rvinecopulib** has the same problem in milder form — 0.18 against 0.23 seconds — for the same reason: threads are cheaper than processes, but they are not free. Neither package should be run on many cores for a problem this small, and that is a statement about the size of the problem rather than about either implementation.

A defect we had to fix before we could measure. The first version of this table showed **VineCopula** getting *slower* as cores were added — at $d = 50$, 52 seconds on one core against 124 on eight. That is a strange enough result to be worth chasing rather than reporting, and it turned out to have three causes, none of them a property of distributing work over processes.

The largest is a captured promise. The tree criterion is built by a helper that takes the candidate family set as an argument, and the closure it returns carries that helper's frame as its environment. Only the AIC and BIC criteria read the family set, so under the default τ the argument is never forced, and the unevaluated promise keeps a reference to the frame of the caller — which holds the data. The criterion is passed to every worker on every dispatch, and **parallel** serializes it once per worker: a function that computes a rank correlation was carrying 7.4 MB, six times the size of the data it was sent alongside. Forcing the argument reduces it to 6 kB. The second is that the parallel mapper was bound to the name `lapply`, shadowing the base function for the remainder of the enclosing function, so two subsequent

		rvinecopulib			VineCopula		
Method	d	1 core	4 cores	8 cores	1 core	4 cores	8 cores
Maximum likelihood	5	2.22	1.18	—	3.68	2.45	—
	10	11.34	4.48	—	17.91	7.73	—
	20	51.81	15.82	—	77.01	26.80	—
τ-inversion	5	0.23	0.15	0.18	0.41	0.98	1.04
	10	1.02	0.40	0.34	1.77	2.11	2.06
	20	4.38	1.38	0.91	7.34	5.49	5.00
	50	28.18	7.87	4.47	46.69	23.27	18.92

Table 8: Structure selection and estimation, fastest of repeated fits, in seconds. Maximum-likelihood rows use $n = 1000$ and τ-inversion rows the full $n = 1509$; an em-dash marks a configuration we did not run. **VineCopula** parallelizes with socket clusters, and the cost of starting them and shipping data to the workers is repaid only when there is enough work per task: under τ-inversion it still loses time at $d = 5$ and $d = 10$, and only becomes profitable by $d = 20$.

list extractions — reading one field out of each of 19,600 records — were dispatched to the worker pool as well; between them they accounted for 57% of the runtime. The third is that the routine computing edge weights received the entire previous tree when it reads seven fields of it, more than half of the object being fitted pair copulas it never looks at.

With all three repaired the $d = 50$ fit on eight cores falls from 80 to 12 seconds, and a 20-dimensional maximum-likelihood fit scales 4.3× on eight cores where it had managed 2.9×. Fitted models are unchanged: we verified that the structure matrix, the families and both parameter matrices are identical to those of the released version, for both estimation methods and at more than one core count. The fixes are contributed upstream (Vatter and Nagler 2026a), and Table 8 reports the repaired version, because a timing against the unrepaired one would measure the defect rather than the design.

We report this at length for two reasons. A reader who reproduces our table against a **VineCopula** release predating the fix will see very different numbers and deserves to know why. And the episode is a caution about benchmarking generally: the original measurement was reproducible, internally consistent, and wrong about the thing it appeared to establish.

Evaluating a fitted model. Fitting is not the only thing one does with a vine, and for many applications it is not the expensive part: a risk calculation simulates from a fitted model repeatedly, and a diagnostic evaluates the Rosenblatt transform. These operations are pure traversals of the h-function cascade, which is exactly what the structure index of Section 3.1 was built for. Table 9 reports them.

The single-core column separates the operations into two groups. Evaluating the density, the log-likelihood or the Rosenblatt transform is 1.7× to 2.4× faster; simulation and the inverse Rosenblatt transform, which are the same computation run backwards through the vine, are a more modest 1.3× to 1.4×. The gap between the two groups is the h-function masks doing their work: a forward traversal can skip the conditional distribution functions no later tree consumes, whereas an inverse traversal needs its arguments in the other order and has less to skip.

	$d = 5$		$d = 10$		$d = 20$		$d = 50$	
Operation	1	8	1	8	1	8	1	8
Density	2.4	8.8	1.9	8.0	2.0	10.9	1.8	11.2
Log-likelihood	2.4	10.3	1.8	7.2	2.0	12.7	2.0	12.1
Rosenblatt	2.2	7.2	1.8	7.6	1.9	8.1	1.7	9.2
Inverse Rosenblatt	1.4	6.3	1.4	6.3	1.4	9.4	1.4	9.3
Simulation	1.4	6.0	1.4	6.5	1.4	9.9	1.3	9.3

Table 9: Evaluation on a fitted model, $n = 1509$: speed-up of **rvinecopulib** over **VineCopula**, on one and on eight cores. Each package is timed on its own fitted model; the two agree on the log-likelihood to a relative 6×10^{-3} or better.

On eight cores the advantage is between 6× and 12.7× and grows with dimension, since the per-observation batches that the threads divide grow with it. Evaluation is where threading pays best: unlike fitting, it has no sequential dependency between trees to respect, so the whole traversal parallelizes over observations. That the two backward operations track each other to within 0.6× in every cell is what one would expect of two operations served by the same traversal, and is a useful check that the numbers are measuring what we think.

Nonparametric pair copulas. The only R implementation of the same estimator is **kdecopula** (Nagler 2018), and the comparison needs two qualifications before any number.

First, the two select the bandwidth by different rules, so they implement the same *method* rather than the same estimator, and their fits differ. We simulated from Clayton copulas and measured the integrated absolute error of the fitted density against the truth, over 200 replications per cell. At Kendall's $\tau = 0.3$ the two are at parity, with **rvinecopulib** ahead by 0.8%, 2.5% and 1.2% at $n = 500$, 1000 and 2000 against Monte Carlo standard errors of about one point. At $\tau = 0.6$ **kdecopula** is clearly the more accurate, and increasingly so with sample size: our error exceeds theirs by 17.4%, 23.9% and 28.3%, with standard errors of 1.3, 1.4 and 1.3. Neither rule is uniformly better and we make no claim that ours is; what matters here is that fitting times are not comparing like with like, and that under strong dependence our bandwidth rule oversmooths.

Second, and more importantly, speed is not the substance of the comparison. **kdecopula** estimates one bivariate copula. It has no vine, no discrete data, no h-function cascade and no way to participate in model selection alongside parametric families. What **rvinecopulib** contributes is not a faster kernel estimator but a nonparametric pair copula that behaves like every other family in the catalogue — it can be selected against parametric candidates, placed at any edge of a vine, and evaluated by the same code as the rest.

With that said, the evaluation timings are worth reporting, because they are not subject to the first qualification: they operate on an already fitted model and measure only the cost of interrogating it. On 10^4 points from a fit with $n = 1000$, **rvinecopulib** evaluates the density in 0.23 ms against 11.3 ms, an h-function in 1.0 ms against 446 ms, and an inverse h-function in 1.3 ms against 7.4 s. The last is a factor of roughly 5600 and is the one that decides whether a nonparametric vine is usable, because inverse h-functions are what simulation and the inverse Rosenblatt transform are built from: a $d = 10$ nonparametric vine needs 45 of them per simulated observation. The mechanism is in Section 3.3 — a closed-form root of a piecewise

quadratic against a bisection that re-integrates the interpolation grid at every step.

What we do not benchmark. Four comparisons are deliberately absent. We do not benchmark discrete or mixed vines, because no alternative implements them and a ratio against nothing is not a result. We do not benchmark against **pyvinecopulib**, because it binds the same engine and any difference measures the language binding rather than the algorithm. We do not time conditional simulation against **VineCopulas** (Claassen *et al.* 2024), the only other implementation that offers it, because it is written in Python: a ratio would confound the language, the implementation and the algorithm, and it is the third a reader would want isolated. And we do not quote the library's internal benchmarks, which compare **vinecopulib** 1.0.0 against its own predecessor rather than against any alternative implementation.

7.4. Where the difference comes from

The differences reported above have three identifiable sources, all described in Section 3, and they explain why the two tracks behave so differently. In density evaluation, simulation, the Rosenblatt transform and estimation at a fixed structure, the precomputed masks of Section 3.1 skip the conditional distribution functions that no later tree consumes. How much that saves depends on the structure — half for a C-vine, about two fifths for a randomly drawn regular vine, and almost nothing for a D-vine in high dimensions — so it contributes to the advantage without accounting for all of it. (During structure selection the structure is not yet known, so the masks cannot apply; there the saving comes from parallelism across candidate edges and from freeing pseudo-observations as soon as their tree has been consumed.) In nonparametric evaluation, the conditional distribution function and its inverse are obtained from cumulative integrals and a closed-form quadratic root rather than by repeated numerical integration and bisection. And in every model containing a Gaussian or Student t pair copula, the normal quantile function is evaluated through the vectorized kernel described at the end of Section 3.2.

8. Summary and limitations

rvinecopulib makes vine copula modeling in R both faster and considerably more general than it has been. The generality comes from three design decisions: variable types are declared once and respected everywhere, so discrete, mixed and zero-inflated data are ordinary rather than special; marginal models enter through protocols rather than a fixed catalogue, so a user's own distribution is a first-class candidate; and every control that governs model selection — the family set, the tree criterion, the truncation level, the estimation method — is exposed rather than fixed. The speed comes from treating a vine structure as an index of the computation it implies: computed once, it tells every algorithm what to compute and what to skip, and it is what makes the parallelism of Section 7.3 effective.

Several limitations are worth stating explicitly.

Model assessment. The package provides no goodness-of-fit test and no Vuong or Clarke test for comparing non-nested models, all of which **VineCopula** offers. Our position is that the derivative machinery of Section 4.7 is the more useful primitive — it yields standard errors, Wald tests and the ingredients of information-matrix tests — but a user who wants a bootstrap goodness-of-fit test will not find one here.

Pair-copula families are closed. Section 5.3 explains why, but the consequence stands: a family not in Table 2 requires a contribution to the C++ library.

The simplifying assumption. Every model in this paper assumes that a conditional pair copula does not depend on the value of its conditioning variables. **rvinecopulib** does not fit non-simplified vines. The `parameters` argument of Section 4.7 allows evaluation of a model whose parameters vary by observation, which is the interface a downstream package implementing conditional dependence needs, but the package does not itself estimate such models.

Nonparametric pair copulas need sample size. A nonparametric pair copula has effective degrees of freedom in the tens, so a d-dimensional vine of them has degrees of freedom in the thousands. Section 6.1 shows the consequence: at $d = 50$ with $n = 1000$ a fully nonparametric vine predicts far worse out of sample than a parametric one. They belong in low dimensions, or with sample sizes that grow with the number of pair copulas. Relatedly, information criteria compare parametric candidates; choosing between a parametric and a nonparametric *model* should be done out of sample.

Derivatives are restricted. Closed-form derivatives are available for continuous, fully parametric models. With discrete variables the backend falls back to central finite differences, which is correct but carries the accuracy of a finite-difference approximation rather than of an analytic formula. For nonparametric pair copulas no derivatives are implemented, so `scores()`, `hessian()` and `vcov()` all refuse a model containing one.

Uncertainty conditions on the margins. The covariance estimates of Section 4.7 treat the marginal transformation as known. This is standard practice, but it is an approximation (Joe 2005; Ko and Hjort 2019), and it errs in the direction that matters: the experiment in Section 4.7 shows the resulting intervals are too short, by about three percentage points of coverage in that design. Because the margin protocols make the marginal estimator a user-supplied function, the correction is not available to the package, and `vcov()` and `confint()` are therefore offered for a fitted copula but not for a complete vine distribution. Quantifying uncertainty in the latter requires resampling both stages.

The algorithms that are new in this release — conditioning-aware structure selection, nonparametric estimation with mixed data, and the derivative cascade in its present form — are developed in a companion paper (Nagler and Vatter 2026). A Python interface to the same engine, with machine-learning ecosystem integration, is described separately (Vatter and Nagler 2026b).

Several R packages already build on **rvinecopulib**: **vinereg** (Nagler 2025d) for D-vine quantile regression (Kraus and Czado 2017), **svines** (Nagler 2025c) for stationary vine time series (Nagler, Krüger, and Min 2022), **portvine** (Sommer 2024) for portfolio risk measures, **copulareg** (Brant and Haff 2022) for copula regression, and **covsim** (Foldnes and Grønneberg 2023) for simulation with prescribed covariance and margins. The observation-specific parameter interface and the marginal protocols were designed with such downstream use in mind.

Computational details

The results in this paper were obtained using R 4.3.3 (R Core Team 2026) with **rvinecopulib** 1.0.0.1.0, which bundles **vinecopulib** 1.0.0. R itself and all packages used are available from CRAN at `https://CRAN.R-project.org/`. **rvinecopulib** is distributed under the GPL-

3; the bundled **vinecopulib** headers are distributed under the MIT license, which is compatible with the GPL. Development takes place at https://github.com/vinecopulib/.

Acknowledgments

[**ACKNOWLEDGMENTS**] We thank the contributors to **vinecopulib** and **rvinecopulib**, and the authors of **VineCopulas** (Claassen *et al.* 2024), whose conditional sampling and structure selection routines we adapted.

References

Aas K, Czado C, Frigessi A, Bakken H (2009). "Pair-Copula Constructions of Multiple Dependence." *Insurance: Mathematics and Economics*, **44**(2), 182–198. doi:10.1016/j.insmatheco.2007.02.001.

Almeida C, Czado C (2012). "Efficient Bayesian Inference for Stochastic Time-Varying Copula Models." *Computational Statistics & Data Analysis*, **56**(6), 1511–1527. doi:10.1016/j.csda.2011.08.015.

Bates D, Eddelbuettel D (2013). "Fast and Elegant Numerical Linear Algebra Using the **RcppEigen** Package." *Journal of Statistical Software*, **52**(5), 1–24. doi:10.18637/jss.v052.i05.

Bedford T, Cooke RM (2001). "Probability Density Decomposition for Conditionally Dependent Random Variables Modeled by Vines." *Annals of Mathematics and Artificial Intelligence*, **32**(1), 245–268. doi:10.1023/A:1016725902970.

Bedford T, Cooke RM (2002). "Vines – A New Graphical Model for Dependent Random Variables." *The Annals of Statistics*, **30**(4), 1031–1068. doi:10.1214/aos/1031689016.

Bevacqua E, Maraun D, Haff IH, Widmann M, Vrac M (2017). "Multivariate Statistical Modelling of Compound Events via Pair-Copula Constructions: Analysis of Floods in Ravenna (Italy)." *Hydrology and Earth System Sciences*, **21**(6), 2701–2723. doi:10.5194/hess-21-2701-2017.

Brant SB, Haff IH (2022). ***copulareg**: Copula Regression*. R package version 0.1.0, URL https://CRAN.R-project.org/package=copulareg.

Brechmann EC, Czado C, Aas K (2012). "Truncated Regular Vines in High Dimensions with Application to Financial Data." *The Canadian Journal of Statistics*, **40**(1), 68–85. doi:10.1002/cjs.10141.

Brechmann EC, Schepsmeier U (2013). "Modeling Dependence with C- and D-Vine Copulas: The R Package **CDVine**." *Journal of Statistical Software*, **52**(3), 1–27. doi:10.18637/jss.v052.i03.

Breiman L, Friedman JH (1985). "Estimating Optimal Transformations for Multiple Regression and Correlation." *Journal of the American Statistical Association*, **80**(391), 580–598. doi:10.1080/01621459.1985.10478157.

Brockwell AE (2007). "Universal Residuals: A Multivariate Transformation." *Statistics & Probability Letters*, **77**(14), 1473–1478. doi:10.1016/j.spl.2007.02.008.

Cambou M, Hofert M, Lemieux C (2017). "Quasi-Random Numbers for Copula Models." *Statistics and Computing*, **27**(5), 1307–1329. doi:10.1007/s11222-016-9688-4.

Chatterjee S (2021). "A New Coefficient of Correlation." *Journal of the American Statistical Association*, **116**(536), 2009–2022. doi:10.1080/01621459.2020.1758115.

Claassen JN, Koks EE, de Ruiter MC, Ward PJ, Jäger WS (2024). "**VineCopulas**: An Open-Source Python Package for Vine Copula Modelling." *Journal of Open Source Software*, **9**(101), 6728. doi:10.21105/joss.06728.

Coblenz M (2021). "**MATVines**: A Vine Copula Package for MATLAB." *SoftwareX*, **14**, 100700. doi:10.1016/j.softx.2021.100700.

Cooke RM, Kurowicka D, Wilson K (2015). "Sampling, Conditionalizing, Counting, Merging, Searching Regular Vines." *Journal of Multivariate Analysis*, **138**, 4–18. doi:10.1016/j.jmva.2015.02.001.

Czado C (2019). *Analyzing Dependent Data with Vine Copulas: A Practical Guide with R*, volume 222 of *Lecture Notes in Statistics*. Springer-Verlag, Cham. doi:10.1007/978-3-030-13785-4.

Czado C, Gneiting T, Held L (2009). "Predictive Model Assessment for Count Data." *Biometrics*, **65**(4), 1254–1261. doi:10.1111/j.1541-0420.2009.01191.x.

Czado C, Jeske S, Hofmann M (2013). "Selection Strategies for Regular Vine Copulae." *Journal de la Société Française de Statistique*, **154**(1), 174–191.

Czado C, Nagler T (2022). "Vine Copula Based Modeling." *Annual Review of Statistics and Its Application*, **9**(1), 453–477. doi:10.1146/annurev-statistics-040220-101153.

Dißmann J, Brechmann EC, Czado C, Kurowicka D (2013). "Selecting and Estimating Regular Vine Copulae and Application to Financial Returns." *Computational Statistics & Data Analysis*, **59**, 52–69. doi:10.1016/j.csda.2012.08.010.

Eddelbuettel D, François R (2011). "**Rcpp**: Seamless R and C++ Integration." *Journal of Statistical Software*, **40**(8), 1–18. doi:10.18637/jss.v040.i08.

Foldnes N, Grønneberg S (2023). ***covsim***: *VITA, IG and PLSIM Simulation for Given Covariance and Marginals*. R package version 1.1.0, URL https://CRAN.R-project.org/package=covsim.

Funk H, Ludwig R, Küchenhoff H, Nagler T (2026). "Towards More Realistic Climate Model Outputs: A Multivariate Bias Correction Based on Zero-Inflated Vine Copulas." *Journal of the Royal Statistical Society C*, **75**(2), 271–302. doi:10.1093/jrsssc/qlaf044.

Gauss J, Nagler T (2026). "Properties of Stepwise Parameter Estimation in High-Dimensional Vine Copulas." *arXiv preprint 2511.17291*, arXiv.org E-Print Archive. doi:10.48550/arXiv.2511.17291.

Geenens G (2014). "Probit Transformation for Kernel Density Estimation on the Unit Interval." *Journal of the American Statistical Association*, **109**(505), 346–358. doi:10.1080/01621459.2013.842173.

Geenens G, Charpentier A, Paindaveine D (2017). "Probit Transformation for Nonparametric Kernel Estimation of the Copula Density." *Bernoulli*, **23**(3), 1848–1873. doi:10.3150/15-BEJ798.

Genest C, Ghoudi K, Rivest LP (1995). "A Semiparametric Estimation Procedure of Dependence Parameters in Multivariate Families of Distributions." *Biometrika*, **82**(3), 543–552. doi:10.1093/biomet/82.3.543.

Guennebaud G, Jacob B, *et al.* (2010). ***Eigen*** *v3*. URL https://eigen.tuxfamily.org/.

Haff IH (2013). "Parameter Estimation for Pair-Copula Constructions." *Bernoulli*, **19**(2), 462–491. doi:10.3150/12-BEJ413.

Hoeffding W (1948). "A Non-Parametric Test of Independence." *The Annals of Mathematical Statistics*, **19**(4), 546–557. doi:10.1214/aoms/1177730150.

Joe H (1996). "Families of m-Variate Distributions with Given Margins and $m(m-1)/2$ Bivariate Dependence Parameters." In L Rüschendorf, B Schweizer, MD Taylor (eds.), *Distributions with Fixed Marginals and Related Topics*, volume 28 of *IMS Lecture Notes–Monograph Series*, pp. 120–141. Institute of Mathematical Statistics, Hayward. doi:10.1214/lnms/1215452614.

Joe H (2005). "Asymptotic Efficiency of the Two-Stage Estimation Method for Copula-Based Models." *Journal of Multivariate Analysis*, **94**(2), 401–419. doi:10.1016/j.jmva.2004.06.003.

Joe H (2014). *Dependence Modeling with Copulas*, volume 134 of *Monographs on Statistics and Applied Probability*. Chapman & Hall/CRC, Boca Raton. doi:10.1201/b17116.

Joe H, Xu JJ (1996). "The Estimation Method of Inference Functions for Margins for Multivariate Models." *Technical Report 166*, Department of Statistics, University of British Columbia. doi:10.14288/1.0225985.

Ko V, Hjort NL (2019). "Model Robust Inference with Two-Stage Maximum Likelihood Estimation for Copulas." *Journal of Multivariate Analysis*, **171**, 362–381. doi:10.1016/j.jmva.2019.01.004.

Kraus D, Czado C (2017). "D-Vine Copula Based Quantile Regression." *Computational Statistics & Data Analysis*, **110**, 1–18. doi:10.1016/j.csda.2016.12.009.

Loader C (1999). *Local Regression and Likelihood*. Statistics and Computing. Springer-Verlag, New York. doi:10.1007/b98858.

Moss J (2025). ***univariateML****: Maximum Likelihood Estimation for Univariate Densities*. R package version 1.5.0, URL https://CRAN.R-project.org/package=univariateML.

Nagler T (2018). "**kdecopula**: An R Package for the Kernel Estimation of Bivariate Copula Densities." *Journal of Statistical Software*, **84**(7), 1–22. doi:10.18637/jss.v084.i07.

Nagler T (2025a). ***kdevine**: Multivariate Kernel Density Estimation with Vine Copulas*. R package version 0.4.6, URL https://CRAN.R-project.org/package=kdevine.

Nagler T (2025b). ***RcppThread**: R-Friendly Threading in C++*. R package version 2.1.2, URL https://CRAN.R-project.org/package=RcppThread.

Nagler T (2025c). ***svines**: Stationary Vine Copula Models*. R package version 0.2.7, URL https://CRAN.R-project.org/package=svines.

Nagler T (2025d). ***vinereg**: D-Vine Quantile Regression*. R package version 0.12.1, URL https://CRAN.R-project.org/package=vinereg.

Nagler T (2025e). ***wdm**: Weighted Dependence Measures*. R package version 0.3.0, URL https://CRAN.R-project.org/package=wdm.

Nagler T (2025f). "Simplified Vine Copula Models: State of Science and Affairs." *Risk Sciences*, **1**, 100022. doi:10.1016/j.risk.2025.100022.

Nagler T, Bumann C, Czado C (2019). "Model Selection in Sparse High-Dimensional Vine Copula Models with an Application to Portfolio Risk." *Journal of Multivariate Analysis*, **172**, 180–192. doi:10.1016/j.jmva.2019.03.004.

Nagler T, Czado C (2016). "Evading the Curse of Dimensionality in Nonparametric Density Estimation with Simplified Vine Copulas." *Journal of Multivariate Analysis*, **151**, 69–89. doi:10.1016/j.jmva.2016.07.003.

Nagler T, Krüger D, Min A (2022). "Stationary Vine Copula Models for Multivariate Time Series." *Journal of Econometrics*, **227**(2), 305–324. doi:10.1016/j.jeconom.2021.11.015.

Nagler T, Schellhase C, Czado C (2017). "Nonparametric Estimation of Simplified Vine Copula Models: Comparison of Methods." *Dependence Modeling*, **5**(1), 99–120. doi:10.1515/demo-2017-0007.

Nagler T, Schepsmeier U, Stöber J, Brechmann EC, Gräler B, Erhardt T (2025). ***VineCopula**: Statistical Inference of Vine Copulas*. R package version 2.6.2, URL https://CRAN.R-project.org/package=VineCopula.

Nagler T, Vatter T (2025). ***kde1d**: Univariate Kernel Density Estimation*. R package version 1.1.1, URL https://CRAN.R-project.org/package=kde1d.

Nagler T, Vatter T (2026). "Algorithms for Vine Copula Models: Conditional Simulation, Nonparametric Estimation with Mixed Data, and Differentiable Likelihoods." *arxiv preprint*, arXiv.org E-Print Archive. In preparation.

Panagiotelis A, Czado C, Joe H (2012). "Pair Copula Constructions for Multivariate Discrete Data." *Journal of the American Statistical Association*, **107**(499), 1063–1072. doi:10.1080/01621459.2012.682850.

Panagiotelis A, Czado C, Joe H, Stöber J (2017). "Model Selection for Discrete Regular Vine Copulas." *Computational Statistics & Data Analysis*, **106**, 138–152. doi:10.1016/j.csda.2016.09.007.

Patton AJ (2006). "Modelling Asymmetric Exchange Rate Dependence." *International Economic Review*, **47**(2), 527–556. doi:10.1111/j.1468-2354.2006.00387.x.

R Core Team (2026). *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria. URL https://www.R-project.org/.

Rényi A (1959). "On Measures of Dependence." *Acta Mathematica Academiae Scientiarum Hungaricae*, **10**(3), 441–451. doi:10.1007/BF02024507.

Rosenblatt M (1952). "Remarks on a Multivariate Transformation." *The Annals of Mathematical Statistics*, **23**(3), 470–472. doi:10.1214/aoms/1177729394.

Schepsmeier U, Stöber J (2014). "Derivatives and Fisher Information of Bivariate Copulas." *Statistical Papers*, **55**(2), 525–542. doi:10.1007/s00362-013-0498-x.

Siek JG, Lee LQ, Lumsdaine A (2002). *The Boost Graph Library: User Guide and Reference Manual*. Addison-Wesley, Boston.

Sklar A (1959). "Fonctions de Répartition à n Dimensions et Leurs Marges." *Publications de l'Institut de Statistique de l'Université de Paris*, **8**, 229–231.

Sommer E (2024). ***portvine**: Vine Based (Un)Conditional Portfolio Risk Measure Estimation*. R package version 1.0.3, URL https://CRAN.R-project.org/package=portvine.

Stöber J, Hong HG, Czado C, Ghosh P (2015). "Comorbidity of Chronic Diseases in the Elderly: Patterns Identified by a Copula Design for Mixed Responses." *Computational Statistics & Data Analysis*, **88**, 28–39. doi:10.1016/j.csda.2015.02.001.

Stöber J, Schepsmeier U (2013). "Estimating Standard Errors in Regular Vine Copula Models." *Computational Statistics*, **28**(6), 2679–2707. doi:10.1007/s00180-013-0423-8.

Tawn JA (1988). "Bivariate Extreme Value Theory: Models and Estimation." *Biometrika*, **75**(3), 397–415. doi:10.1093/biomet/75.3.397.

Theußl S, Schwendinger F, Hornik K (2020). "**ROI**: An Extensible R Optimization Infrastructure." *Journal of Statistical Software*, **94**(15), 1–64. doi:10.18637/jss.v094.i15.

Vatter T, Chavez-Demoulin V (2015). "Generalized Additive Models for Conditional Dependence Structures." *Journal of Multivariate Analysis*, **141**, 147–167. doi:10.1016/j.jmva.2015.07.003.

Vatter T, Nagler T (2018). "Generalized Additive Models for Pair-Copula Constructions." *Journal of Computational and Graphical Statistics*, **27**(4), 715–727. doi:10.1080/10618600.2018.1451338.

Vatter T, Nagler T (2025). "Throwing Vines at the Wall: Structure Learning via Random Search." *arXiv preprint 2510.20035*, arXiv.org E-Print Archive. doi:10.48550/arXiv.2510.20035.

Vatter T, Nagler T (2026a). "Cluster Cleanup and Parallelization Fixes for **VineCopula**." Pull requests 104 and 105, https://github.com/tnagler/VineCopula. Merged upstream.

Vatter T, Nagler T (2026b). "**pyvinecopulib**: Scalable Vine Copula Modeling for Python and Machine Learning." *Technical report*, Submitted.

White H (1982). "Maximum Likelihood Estimation of Misspecified Models." *Econometrica*, **50**(1), 1–25. doi:10.2307/1912526.

Wilson DB (1996). "Generating Random Spanning Trees More Quickly Than the Cover Time." In *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing (STOC '96)*, pp. 296–303. ACM, New York. doi:10.1145/237814.237880.

Wolny-Dominiak A, Trzesiok M (2014). ***insuranceData**: A Collection of Insurance Datasets Useful in Risk Classification*. R package version 1.0, URL https://CRAN.R-project.org/package=insuranceData.

Zeileis A (2006). "Object-Oriented Computation of Sandwich Estimators." *Journal of Statistical Software*, **16**(9), 1–16. doi:10.18637/jss.v016.i09.

Affiliation:

Thomas Nagler
Department of Statistics
Ludwig-Maximilians-Universität München
Ludwigstraße 33, 80539 München, Germany
E-mail: thomas.nagler@stat.uni-muenchen.de

Thibault Vatter
Geneva School of Business Administration
HES-SO University of Applied Sciences and Arts Western Switzerland
Rue de la Tambourine 17, 1227 Carouge, Switzerland
E-mail: thibault.vatter@hesge.ch
URL: https://vinecopulib.github.io/rvinecopulib/

Journal of Statistical Software https://www.jstatsoft.org/
published by the Foundation for Open Access Statistics https://www.foastat.org/
MMMMMM YYYY, Volume VV, Issue II *Submitted:* yyyy-mm-dd
doi:10.18637/jss.v000.i00 *Accepted:* yyyy-mm-dd